

学校代码:  
分类号: O643.12  
密级: 公开  
UDC: 544.4  
学号:

# 硕士学位论文

|            |                               |
|------------|-------------------------------|
| 论文题目       | 支持混合隔离级别的<br>分布式键值数据库原型的设计与实现 |
| 作者姓名       | ×××                           |
| 专业学位类别(领域) | 工程硕士(软件工程领域)                  |
| 研究方向       | 软件工程                          |
| 导师姓名       | ××× ×××                       |

2026年3月20日

答辩委员会主席\_\_\_\_\_

评 阅 人\_\_\_\_\_

\_\_\_\_\_

\_\_\_\_\_

\_\_\_\_\_

\_\_\_\_\_

论文答辩日期 2026 年 5 月 20 日

研究生签名：

导师签名：

# **Design and Implementation of a Mixed-isolation-level Distributed Key-Value Database Prototype**

by

× × ×

Supervised by

× × ×   × × ×

A dissertation submitted to

the graduate school of

in partial fulfilment of the requirements for the degree of

MASTER OF ENGINEERING

in

Master of Engineering (Software Engineering)

Software Institute

March 20, 2026



# 研究生毕业论文中文摘要首页用纸

毕业论文题目：支持混合隔离级别的分布式键值数据库原型的设计与实现

工程硕士(软件工程领域)专业 2024 级硕士生姓名：×××

指导教师(姓名、职称)：××× ×××

## 摘 要

分布式数据库已成为现代互联网应用迈向分布式与云原生架构的重要基础设施。面对高并发和多样化业务需求，系统设计长期需要在数据一致性与执行性能之间进行权衡：单一强隔离级别能够提供严格的正确性保障，但往往带来较高的并发控制开销；单一弱隔离级别虽然吞吐量较高，却难以满足关键事务的严格一致性要求。

针对上述问题，本文围绕混合隔离级别事务的统一语义建模与系统实现展开研究，设计并实现了一个基于分片（Sharding）架构、支持混合隔离级别的分布式键值数据库原型。本文提出了一种面向混合隔离级别场景的并发控制协议，使事务能够根据业务需求选择不同隔离级别，并支持跨分片事务的原子执行。

为保证协议在混合场景下的正确性，本文使用  $TLA^+$  对系统操作语义进行形式化建模，并通过模型检查验证关键安全属性；在此基础上，进一步构建了一个分布式数据库验证平台，实现了分布式事务调度、多版本并发控制（MVCC）存储引擎以及网络通信模块，用于模拟分布式环境下的事务处理流程。

基准测试结果表明，本文设计的混合隔离级别机制能够正确执行混合事务逻辑；相较于统一采用强隔离级别的基线配置，该机制在保证目标语义成立的前提下能够稳定提升系统吞吐量并改善平均延迟。上述结果说明，本文提出的机制具备较好的工程可行性，并为在分布式数据库中兼顾正确性与性能提供了一种可行路径。

**关键词：**数据库；事务；混合隔离级别； $TLA^+$ ；形式化验证



# 研究生毕业论文英文摘要首页用纸

THESIS: Design and Implementation of a Mixed-isolation-level Distributed Key-Value Database Prototype

SPECIALIZATION: Master of Engineering (Software Engineering)

POSTGRADUATE: \_\_\_\_\_ x x x

MENTOR:                     XXX  XXX                    

## ABSTRACT

Distributed databases have become essential infrastructure for modern internet applications moving toward distributed and cloud-native architectures.

Under high concurrency and diverse workloads, system design must continually balance data consistency against execution performance: a single strong isolation level provides strict correctness guarantees but often incurs substantial concurrency-control overhead, whereas a single weak isolation level may deliver higher throughput but cannot satisfy the stringent consistency requirements of critical transactions.

To address this problem, this thesis studies unified semantic modeling and system implementation for transactions running under mixed isolation levels. We design and implement a prototype distributed key-value database based on a sharding architecture that supports mixed isolation levels, and propose a concurrency control protocol tailored to such scenarios. The protocol allows transactions to select different isolation levels according to application requirements while supporting atomic execution of cross-shard transactions.

To ensure correctness in mixed-isolation settings, we model the system semantics using TLA<sup>+</sup> and verify key safety properties through model checking. Based on the verified model, we further build a distributed database validation platform that implements distributed transaction scheduling, a multi-version concurrency control (MVCC) storage engine, and network communication modules for realistic simulation of distributed transaction processing.

Benchmark results show that the proposed mechanism executes mixed transaction

## ABSTRACT

---

logic correctly and, compared with a baseline configuration that uniformly adopts a strong isolation level, can consistently improve system throughput and reduce average latency while preserving the required semantic guarantees. These results demonstrate the engineering feasibility of the proposed mechanism and suggest a practical way to better balance correctness and performance in distributed database systems.

**KEYWORDS:** Database; Transaction; Mixed Isolation Levels; TLA<sup>+</sup>; Formal Verification

# 目 录

|                       |    |
|-----------------------|----|
| 目 录                   | V  |
| 插图目录                  | IX |
| 表格目录                  | XI |
| 第一章 绪论                | 1  |
| 1.1 研究背景与研究意义         | 1  |
| 1.2 研究现状与挑战           | 3  |
| 1.2.1 研究现状            | 3  |
| 1.2.2 研究挑战            | 4  |
| 1.3 本文主要工作与贡献         | 5  |
| 1.4 本文组织结构            | 6  |
| 第二章 背景知识与相关技术         | 9  |
| 2.1 数据库建模             | 9  |
| 2.2 异常                | 13 |
| 2.3 公理和隔离级别           | 16 |
| 2.3.1 公理              | 16 |
| 2.3.2 隔离级别            | 17 |
| 2.4 $TLA^+$ 与 TLC 基础  | 20 |
| 2.5 本章小结              | 23 |
| 第三章 面向混合隔离级别的系统操作语义设计 | 25 |
| 3.1 混合隔离级别问题          | 25 |
| 3.2 系统架构建模            | 27 |

|                                              |                             |           |
|----------------------------------------------|-----------------------------|-----------|
| 3.3                                          | 混合逻辑时钟                      | 32        |
| 3.4                                          | 面向混合隔离级别的操作语义               | 33        |
| 3.5                                          | 本章小结                        | 41        |
| <b>第四章 支持混合隔离级别机制的 TLA<sup>+</sup> 建模与验证</b> |                             | <b>43</b> |
| 4.1                                          | 事务操作语义的 TLA <sup>+</sup> 表达 | 43        |
| 4.2                                          | 公理和隔离级别约束                   | 43        |
| 4.2.1                                        | 状态变量与动作定义                   | 43        |
| 4.2.2                                        | 抽象执行关系与公理约束                 | 46        |
| 4.3                                          | 模型检查实验与结果                   | 47        |
| 4.4                                          | 本章小结                        | 49        |
| <b>第五章 分布式键值数据库原型的架构设计与实现</b>                |                             | <b>51</b> |
| 5.1                                          | 原型系统概述                      | 51        |
| 5.2                                          | 需求分析                        | 51        |
| 5.2.1                                        | 功能需求                        | 51        |
| 5.2.2                                        | 非功能需求                       | 52        |
| 5.2.3                                        | 用例分析                        | 53        |
| 5.3                                          | 系统设计                        | 53        |
| 5.3.1                                        | 总体架构设计                      | 54        |
| 5.3.2                                        | 核心模块设计                      | 55        |
| 5.4                                          | 系统实现                        | 56        |
| 5.4.1                                        | 存储分片与分布式事务                  | 56        |
| 5.4.2                                        | 混合逻辑时钟                      | 59        |
| 5.4.3                                        | 通信原理与同步机制                   | 60        |
| 5.4.4                                        | 用户客户端和测试客户端                 | 61        |
| 5.4.5                                        | 系统日志与操作可观测性                 | 64        |
| 5.5                                          | 本章小结                        | 65        |
| <b>第六章 基于分布式键值数据库原型的系统测试</b>                 |                             | <b>67</b> |
| 6.1                                          | 测试环境概述                      | 67        |

---

|                                                |           |
|------------------------------------------------|-----------|
| 6.2 测试负载与评价指标 . . . . .                        | 68        |
| 6.3 测试结果与分析 . . . . .                          | 69        |
| 6.4 本章小结 . . . . .                             | 71        |
| <b>第七章 工作总结与展望 . . . . .</b>                   | <b>73</b> |
| 7.1 工作总结 . . . . .                             | 73        |
| 7.2 未来展望 . . . . .                             | 74        |
| <b>参考文献 . . . . .</b>                          | <b>77</b> |
| <b>致谢（盲审阶段，暂时隐去） . . . . .</b>                 | <b>81</b> |
| <b>附录 A TLA<sup>+</sup> 模型检查补充材料 . . . . .</b> | <b>83</b> |
| A.1 SmallBank 负载测试用例 . . . . .                 | 83        |



## 插图目录

|     |                                      |    |
|-----|--------------------------------------|----|
| 2-1 | 因果关系违背示例 . . . . .                   | 14 |
| 2-2 | 破碎读示例 . . . . .                      | 14 |
| 2-3 | 脏读示例 . . . . .                       | 15 |
| 2-4 | 更新丢失示例 . . . . .                     | 15 |
| 2-5 | 长分叉异常示例 . . . . .                    | 15 |
| 2-6 | 写偏斜示例 . . . . .                      | 16 |
| 2-7 | 隔离级别间的偏序关系 <sup>[18]</sup> . . . . . | 19 |
| 2-8 | Client_Server 规约 . . . . .           | 21 |
| 3-1 | 混合隔离级别下抽象执行判定示例 . . . . .            | 27 |
| 4-1 | 关系 VIS 和 AR 的形式化表达 . . . . .         | 46 |
| 4-2 | 公理的形式化表达 . . . . .                   | 47 |
| 4-3 | TLC 模型检查器的终端输出示例 . . . . .           | 49 |
| 5-1 | 系统用例图 . . . . .                      | 53 |
| 5-2 | 系统总体架构图 . . . . .                    | 54 |
| 5-3 | 组件交互时序图 . . . . .                    | 57 |
| 5-4 | 两阶段提交可见性漏洞示意图 . . . . .              | 58 |
| 5-5 | 组件交互时序图 . . . . .                    | 59 |
| 5-6 | 长连接配置同步示意图 . . . . .                 | 61 |
| 5-7 | 用户客户端示例交互 . . . . .                  | 62 |
| 5-8 | ClientPool 机制示意图 . . . . .           | 63 |
| 5-9 | 测试用例示例 . . . . .                     | 64 |
| 6-1 | 系统资源水位与响应延迟变化图 . . . . .             | 68 |

|     |                                        |    |
|-----|----------------------------------------|----|
| 6-2 | 不同部署方式下各隔离级别配置的吞吐量对比 . . . . .         | 69 |
| 6-3 | 不同部署方式下各策略相对 SER 的吞吐提升百分比 . . . . .    | 70 |
| 6-4 | 不同部署方式下 SER 与 MIAED 的事务回滚率对比 . . . . . | 71 |
| A-1 | SmallBank 负载测试用例 . . . . .             | 83 |

## 表格目录

|     |                            |    |
|-----|----------------------------|----|
| 2-1 | 一致性模型及其所满足的公理与异常 . . . . . | 19 |
| 3-1 | 隔离级别及其公理组合 . . . . .       | 34 |
| 3-2 | 隔离级别实现的核心定制点 . . . . .     | 40 |
| 6-1 | 本地测试环境配置 . . . . .         | 67 |



# 第一章 绪论

## 1.1 研究背景与研究意义

在互联网应用持续演进的背景下，数据库系统已经成为信息基础设施中的核心组成部分，几乎支撑着所有关键业务系统的运行<sup>[1]</sup>。从在线交易、社交网络到云计算与人工智能应用，海量数据的高效存储、实时访问与可靠管理，直接影响上层业务的性能、可用性与安全性<sup>[2-3]</sup>。随着互联网业务规模持续扩大，分布式和云原生架构快速普及，传统集中式数据库在扩展性、可用性以及运维成本等方面逐渐暴露出难以克服的瓶颈，对于满足大规模在线服务与大数据业务需求显得愈发吃力<sup>[4]</sup>。分布式数据库正是在这样的背景下应运而生。通过在多节点上对数据进行分片与复制，分布式数据库能够实现水平扩展与高可用，已成为支撑大规模在线服务与数据密集型业务的重要技术路径<sup>[5]</sup>。与此同时，随着业务系统从单体应用逐步演化为微服务和跨地域部署形态，数据访问路径、事务边界以及系统协同关系也变得更加复杂<sup>[6-7]</sup>，这进一步放大了数据库系统在一致性保障与性能优化方面的设计压力。

事务（Transaction）是数据库系统中保障数据一致性与隔离性的核心机制，也是支撑上层业务正确性的基础<sup>[8]</sup>。在数据库系统中，事务被视为一个逻辑工作单元，由一系列操作构成，其目标是在并发访问和系统故障存在的条件下，仍保持数据的完整性与一致性。经典数据库理论通常以 ACID 特性来描述事务的基本要求<sup>[9-10]</sup>：

- **原子性（Atomicity）**：事务中的所有操作要么全部执行成功，要么全部不执行。若执行过程中发生错误，系统必须回滚到事务开始前的状态，避免出现部分完成的中间状态。
- **一致性（Consistency）**：事务执行前后，数据库应当从一个一致状态转换到另一个一致状态，不能破坏主键、外键或业务规则等完整性约束。
- **隔离性（Isolation）**：多个事务并发执行时，每个事务都不应受到其他事务

中间状态的干扰，系统需要通过并发控制机制保证结果与某种约定的顺序执行相一致。

- **持久性 (Durability):** 一旦事务提交，其对数据库的修改应被永久保存，即使系统发生故障也不应丢失。

然而，在分布式环境中，实现事务语义的代价显著提高。由于数据分布在多个物理节点上，网络延迟、节点故障与分区容错等问题使严格的一致性保障变得复杂且昂贵。因此，分布式数据库往往需要在一致性、可用性与分区容错之间进行权衡<sup>[11-12]</sup>。在这一背景下，隔离级别设计逐渐成为系统在正确性与性能之间进行取舍的重要手段。

从应用视角看，不同业务对隔离语义的敏感程度并不一致<sup>[13]</sup>。金融转账、订单结算等场景通常要求事务结果具备较强的一致性保证，任何异常读写都可能引发显著的业务风险；而日志分析、推荐统计、在线监控等场景往往更强调吞吐量、响应时间与系统扩展能力，对隔离强度的要求相对宽松。如果系统只能为所有事务统一提供同一种隔离级别<sup>[14]</sup>，就容易出现两类问题：要么为了保证少量关键事务而让整体系统承担过高的同步开销，要么为了追求吞吐量而使核心业务暴露在不必要的一致性风险之下。

对分布式场景中的事务处理而言，单一隔离级别往往难以同时满足所有业务需求。混合隔离级别 (Mixed Isolation Level, MIL) 允许不同事务根据自身业务语义与性能需求，在同一系统中运行于不同隔离级别之下<sup>[15-16]</sup>。例如，对一致性要求极高的转账事务可以采用可串行化隔离级别，而对性能敏感、但允许一定语义放宽的日志分析事务则可以采用较弱隔离级别。这样，系统就有机会在保障关键事务正确性的同时，释放其他事务的并发潜力。

当前一些数据库产品已经支持“不同事务可配置不同隔离级别”的工程能力<sup>[17]</sup>，但这并不等同于混合隔离级别已经具备统一、清晰且可验证的语义规范。现有理论基础，例如基于冲突和视图可串行化的理论，通常默认所有事务运行于同一隔离级别之下；在多版本并发控制 (MVCC) 环境中，不同可见性规则事务之间的交互行为，例如 SER 与 SI、RR 等级别之间的组合，仍缺乏统一而完备的形式化描述<sup>[18]</sup>。近期 Bouajjani 等人的工作<sup>[19]</sup>更多从黑盒角度测试数据库对隔离级别的保证，而如何建立面向系统内部机制的白盒分析方法和形式化定义，仍然是值得进一步研究的问题。

基于上述背景，本文关注三个彼此关联的问题：如何为混合隔离级别事务建立统一语义模型，如何把该模型转化为可执行、可验证的系统操作语义，以及如何借助原型实现与实验评估说明该机制在工程上确实可行并具备性能收益。对这些问题的回答，既关系到混合隔离级别的理论边界，也关系到相关系统能否真正落地。

## 1.2 研究现状与挑战

### 1.2.1 研究现状

围绕分布式事务、隔离级别与形式化验证的研究，现有工作已经在事务理论、并发控制、多版本机制以及一致性建模等方面积累了较为扎实的基础<sup>[8-9,15-16,20]</sup>。然而，面向混合隔离级别场景的研究仍然相对不足，尤其在统一语义定义、形式化验证方法以及理论与实现衔接方面仍存在明显空缺。

首先，现有研究对混合隔离级别的语义刻画仍不充分。传统隔离级别理论大多建立在“所有事务运行于同一隔离级别”的前提之上，其核心目标是描述单一隔离策略下允许或禁止的异常行为。然而，在混合隔离级别场景中，不同事务依据不同可见性规则并发执行，事务之间的依赖关系、冲突传播路径以及可接受执行结果的判定条件都会显著复杂化。现有研究虽然已经开始关注这一问题，但对于不同隔离级别事务交互时的统一语义定义、异常边界及其判定准则，尚未形成充分一致的理论框架<sup>[19]</sup>。

其次，现有研究在形式化验证层面仍存在不足。已有工作通常能够针对特定一致性模型、特定隔离级别或局部并发控制机制进行分析与证明，但对于混合隔离级别下分布式事务系统的整体行为，尤其是跨分片事务执行过程中状态演化、版本可见性与提交约束之间的联动关系，仍缺乏系统性的形式化表达与验证方法。这使得很多系统设计只能停留在经验性论证或局部正确性分析层面，难以对复杂并发场景下的关键安全性质给出严格保证。

再次，现有研究在理论模型与工程实现之间仍存在明显断层<sup>[21]</sup>。部分研究侧重一致性公理、异常定义或抽象执行模型，能够从理论层面解释并发行为；另一些工作则聚焦数据库系统实现，通过事务协调、时间戳定序、MVCC 或两阶段提交等机制提升性能与一致性保障能力。然而，两者之间往往缺乏稳定、清晰

的映射关系，即理论模型难以直接指导系统实现，而工程系统中的关键设计决策也缺少可回溯的语义支撑。这种脱节限制了研究成果的可验证性、可复现性与可推广性。

最后，现有研究对混合隔离级别系统的实验评估仍不够系统。许多工作主要围绕单一隔离级别、单类负载或特定系统设定展开性能测试，对于不同隔离级别混合配置下系统吞吐量、事务延迟、一致性风险与扩展能力之间的关系缺乏更系统的实验分析。特别是在跨分片事务、复杂读写依赖以及多种业务负载并存的条件下，如何定量评估混合隔离级别机制在正确性与性能之间的权衡效果，仍然是当前研究中的薄弱环节。

从工程实践角度看，现有主流数据库产品在分布式事务处理与混合隔离级别支持方面也体现出不同的设计路径。传统关系型数据库如 MySQL 与 PostgreSQL 具备成熟的 ACID 能力，但其分布式事务通常依赖 XA 或相关协调机制，在跨节点场景下容易遭遇较高的协调开销<sup>[22-23]</sup>。Redis 等高性能键值存储系统强调水平扩展与高吞吐，但其事务能力主要局限于单分片操作<sup>[24]</sup>。FoundationDB、Google Spanner 和 TiKV 等 NewSQL 系统则提供了跨节点强一致事务处理能力<sup>[25]</sup>，但其并发控制协议通常建立在统一隔离级别假设之上，对混合隔离级别的统一语义支持仍不充分。以 Cassandra 与 DynamoDB 为代表的 AP 型数据库则更强调高可用与分区容错，在强一致事务处理方面存在架构层面的限制<sup>[26-28]</sup>。这说明，现有系统虽然在不同方向上分别取得了工程成功，但对于“多种隔离级别如何在同一分布式系统中协同成立”这一问题，仍缺少兼顾语义严谨性与实现可行性的统一回答<sup>[29]</sup>。

总体来看，现有研究已经为分布式事务处理和一致性保障奠定了基础，但放到混合隔离级别场景中，仍然存在语义定义不统一、形式化验证不足、理论与实现衔接偏弱以及实验评估不够系统等问题。基于这一判断，本文尝试把统一语义建模、形式化验证、原型实现和实验评估放在同一条研究链路中讨论。

### 1.2.2 研究挑战

基于上述研究现状，本文将问题收束为以下三个彼此衔接的关键挑战。

首先，混合隔离级别缺乏统一语义模型。现有隔离级别理论大多建立在所有事务运行于同一隔离级别的前提之上，而在混合隔离级别场景中，不同可见性规

则的事务并发执行，会使事务间的依赖关系、冲突传播路径以及可接受执行结果的判定条件显著复杂化。因此，如何为不同隔离级别事务的并发交互建立统一、严谨的语义框架，是本文首先要解决的理论问题。

其次，统一语义缺乏面向系统行为的形式化验证手段。现有形式化方法通常针对特定一致性模型或局部并发控制机制进行分析，对于混合隔离级别下分布式事务系统的整体行为，尤其是跨分片事务执行过程中状态演化、版本可见性与提交约束之间的联动关系，仍缺乏系统性的形式化表达与验证方法。因此，如何构建覆盖混合隔离级别全局行为的形式化验证框架，是本文的第二个关键问题。

最后，形式化语义与工程实现之间仍存在明显断层。抽象的一致性公理与异常定义难以直接映射到具体的并发控制机制与系统实现，而工程系统中的关键设计决策也往往缺少可回溯的语义支撑。进一步地，混合隔离级别配置下不同事务对并发控制开销的要求并不相同，系统还需要在正确性与性能之间取得平衡。因此，如何建立理论模型、协议设计、系统实现与性能评估之间的稳定对应关系，是本文需要回答的工程落地问题。换言之，只有当语义定义能够真正约束协议行为、协议设计能够真实落入系统实现、而系统实现又能够通过实验结果回应最初的研究问题时，混合隔离级别机制才具有完整而可信的研究价值。

围绕这三个挑战，本文分别从统一语义建模、形式化验证、原型系统实现与实验评估四个层面展开研究，使各部分能够相互对应，而不是彼此分离。

### 1.3 本文主要工作与贡献

本文采用分片（Sharding）架构<sup>[30]</sup>，参考 MongoDB 的设计思想，将数据水平切分并分布存储在多个物理节点上，以实现存储容量与处理能力的水平扩展。系统包含 Router、Shard 和 Config Server 等核心组件，其中 Router 作为客户端请求的统一入口，负责根据分片键将读写请求路由到对应节点；Config Server 负责维护全局元数据和分片映射关系。在此基础上，本文围绕混合隔离级别场景下跨分片事务处理的语义定义、正确性验证与系统实现展开研究。

本文的主要工作如下：

本文首先面向分布式键值数据库场景，构建了混合隔离级别事务处理的统一语义描述框架。该框架从事务可见性、读写冲突约束以及异常边界等方面，对

不同隔离级别事务在并发执行过程中的行为进行了系统刻画，为后续操作语义设计、正确性分析和系统实现提供了统一的理论基础。

在此基础上，本文进一步设计了支持混合隔离级别的分布式事务操作语义，并给出了与系统实现相对应的关键状态转换过程和并发控制机制。通过将事务执行、版本演化、提交协调与跨分片交互纳入统一描述，本文建立了从抽象语义到具体系统机制之间的映射关系，从而增强了设计方案的可实现性与可分析性。

为验证所提出方法的正确性，本文使用  $TLA^+$  及 TLC 对事务操作语义、一致性公理和隔离级别约束进行了形式化建模与模型检查，分析系统在给定模型范围内是否满足预期的语义要求。 $TLA^+$  在形式化验证领域得到广泛应用<sup>[31]</sup>。这部分工作一方面提高了关键设计结论的可信度，另一方面也为后续原型实现提供了可追溯的依据。

在工程实现与实验评估方面，本文设计并实现了一个支持混合隔离级别的分布式键值数据库原型系统，完成了路由、分片、MVCC、HLC、跨分片事务协调及可观测性模块的工程实现。在此基础上，本文进一步通过基准实验评估不同隔离级别配置下的系统吞吐量与延迟表现。实验结果表明，混合隔离级别机制能够在保证事务满足目标语义的前提下，有效提升系统整体执行效率，体现出较好的工程应用价值。

### 1.4 本文组织结构

第一章为绪论，主要介绍研究背景与研究意义，分析相关研究现状与面临的关键挑战，并概述本文的主要工作、不足与组织结构。

第二章为相关技术与理论，介绍数据库异常与隔离级别、一致性建模基础以及  $TLA^+$  与 TLC 等形式化验证相关技术，为后续章节奠定理论基础。

第三章讨论系统操作语义，从系统架构、混合逻辑时钟与事务操作语义三个方面给出支持混合隔离级别的分布式事务执行模型。

第四章围绕事务操作语言的  $TLA^+$  表达、一致性公理与隔离级别约束以及模型检查实验与结果展开，对第三章提出的操作语义进行形式化验证。

第五章介绍原型系统的架构设计与系统实现，重点说明关键模块的设计目标、实现机制与工程落地方式。

第六章通过实验验证混合隔离级别机制对系统性能的提升效果，分析不同部署拓扑与不同隔离级别配置下的吞吐量表现。

第七章对全文工作进行总结，并展望未来的研究方向。



## 第二章 背景知识与相关技术

本章围绕分布式数据库中的事务一致性与隔离语义，对后文所需的核心概念、形式化工具与判定基础进行重新整理。与其先给出抽象定义，不如先从几个容易理解的案例出发：当并发事务分别在不同节点、不同时间观察同一组数据时，究竟会出现什么异常；而数据库通过提升隔离级别，又是在禁止哪些异常、保留哪些并发能力。围绕这一问题，本章按照“例子、数据库建模、异常、公理与隔离级别、 $TLA^+$  与  $TLC$ ”的顺序展开：首先用典型案例引出事务异常与隔离需求；其次给出统一的数据库建模框架；随后对典型异常及其对应的公理与隔离级别进行形式化刻画；最后介绍本文后续验证所依赖的  $TLA^+$  与  $TLC$ ，并说明如何基于这些工具判定一个执行是否满足给定隔离级别。

### 2.1 数据库建模

理解事务语义，一个更直接的入口不是先记忆抽象定义，而是先观察“错误结果是怎样发生的”。数据库异常描述的是并发事务之间可见性关系和提交顺序的不一致：某个事务看到了本不该看到的数据，或没有看到它本应看到的数据，又或者多个事务各自得到“局部合理、整体错误”的结果。正因为这些异常可能实际出现，数据库系统才需要通过一致性公理和隔离级别去限制它们。

为了便于说明，下面给出一个典型案例。

设事务  $txn_1$  发布一条帖子，事务  $txn_2$  读取帖子后写入评论，事务  $txn_3$  则同时读取帖子与评论：

$$\begin{aligned} & txn_1 \{ \text{write}(x, \text{post}); \text{write}(y, \text{null}) \} \\ & \parallel txn_2 \{ u = \text{read}(x); \text{write}(y, \text{comment}) \} \\ & \parallel txn_3 \{ v = \text{read}(x); w = \text{read}(y) \} \end{aligned} \tag{2-1}$$

若系统按串行顺序执行，则  $txn_3$  读到评论时，也一定能读到对应的帖子。然

而在分布式环境中,  $txn_1$  的更新可能先传播到执行  $txn_2$  的节点, 却尚未传播到执行  $txn_3$  的节点, 于是出现  $v = \text{null}, w = \text{comment}$  的结果。这个结果并不违单次读写语法, 却违背了“评论依赖帖子”的因果关系。由此可以看出, 仅保证单事务内部的读写正确还不够; 系统还需要约束跨事务的可见性传播。较弱的隔离级别往往允许这类异常, 而更强的隔离级别会显式禁止它。

由上述案例可以看出, 数据库异常并不是彼此孤立的“反例集合”, 而是理解事务一致性问题的切入点。分布式数据库中的问题并不仅仅来自“多个事务同时运行”, 更来自“不同节点在不同时间看到不同版本”。在单机环境中, 我们更容易把事务理解为围绕一个共享内存状态展开; 而在分布式系统中, 数据通常分散在多个分片、副本和节点上, 事务的读写效果需要经过网络传播、日志复制和提交协议, 才会逐步对其他事务可见。因此, “系统此刻的状态是什么”并不是一个天然唯一的答案, 不同事务观察到的状态也未必一致。

这正是数据库建模的必要性所在。所谓数据库模型, 并不是简单地给数据库贴上“强”或“弱”的标签, 而是对以下问题作出精确定义: 一个事务可以看到哪些已提交事务; 多个事务的写入如何在逻辑上排序; 面对写冲突、因果依赖和历史分叉时, 系统必须遵守哪些约束。只有把这些问题明确下来, 后续的系统设计、协议实现与形式化验证才有共同语义基础。

从工程角度看, 数据库建模是在正确性与性能之间建立契约。若希望事务行为尽可能接近串行执行, 就需要引入更强的同步、验证或加锁机制, 代价是吞吐下降和延迟上升; 若希望保留更多本地并发和异步复制能力, 就需要容忍某些异常在边界条件下出现。因此, 建模的目的并非简单追求“越强越好”, 而是明确数据库系统向应用层公开的语义承诺: 系统保证哪些行为不会发生, 应用又需要承担哪些剩余风险。

对本文而言, 数据库建模还有一个直接目的, 即为后文的混合隔离级别机制提供统一描述语言。由于本文关注的不是单一隔离级别, 而是一个系统中不同事务可能采用不同隔离级别的场景, 因此必须先建立一套能够同时刻画事务、历史、可见性关系与全局排序关系的抽象框架。只有在这一框架内, 才能进一步讨论“某个事务满足 SI, 而另一个事务满足 SER”时, 整个执行是否仍然成立。

为获得统一且可推理的描述, 本文从事务、操作、快照、历史和抽象执行五个层次进行建模。

从全局视角出发，使用符号  $H$  (history) 表示事务执行历史。历史由若干事务 (transaction, 记为  $T$  或  $T_x$ ) 组成，并为每个事务标注其所采用的隔离级别 (isolation level)。本文讨论的六个典型隔离级别为：可串行化 (Serializable, SER)、快照隔离 (Snapshot Isolation, SI)、前缀一致性 (Prefix Consistency, PC)、并发快照隔离 (Parallel Snapshot Isolation, PSI)、因果一致性 (Causal Consistency, CC) 和读原子性 (Read Atomic, RA)<sup>[20]</sup>。后文将基于统一公理体系给出它们的形式化定义。

从事务内部看，一个事务  $T$  被视为一组操作 (Operation) 的有序执行序列。由于本文主要面向键值数据库，暂不考虑复杂谓词读等扩展操作，因此将事务中的基本操作抽象为以下五类：

- **start**: 事务开始操作，表示一个新事务被创建；
- **commit**: 事务提交操作，表示事务成功结束并使写入生效；
- **abort**: 事务中止操作，表示事务执行失败并回滚；
- **read( $k, v$ )**: 读操作，表示读取键  $k$  并得到值  $v$ ；
- **write( $k, v$ )**: 写操作，表示将键  $k$  更新为值  $v$ 。

其中，**read()** 和 **write()** 的操作对象均为键  $k$ ，所有键的集合记为  $\mathcal{K}$ 。

为了描述“事务在读取时看到的数据库状态”，本文进一步引入快照 (snapshot, 记为  $S$ )。快照可以理解为事务在某一逻辑时刻观察到的只读一致性视图，是多版本并发控制 (MVCC) 中的基础概念。直观地说，事务并不直接读取数据库的“当前唯一值”，而是在自己的可见范围内选择每个键的一个版本；形式化地，本文用快照函数  $S$  将键映射到值，表示给定视图下各键的取值。

基于以上对象，本文用二元组描述“集合 + 顺序”的结构。事务  $T$  定义为  $(O, po)$ ，其中  $O \subseteq Op$  为事务包含的操作集合， $po \subseteq O \times O$  为程序顺序 (program order)，用于描述事务内部操作的先后关系。类似地，执行历史  $H$  中还包含会话顺序  $SO$  (session order)，用于刻画同一会话中事务的先后次序。

以上概念的形式化定义如下：

$$\begin{aligned}
\mathcal{K} &= \{x, y, \dots\}, \quad \mathcal{V} = \{\perp, 1, 2, \dots, n\} \\
Op &= \{\text{start}, \text{commit}, \text{abort}\} \cup \{\text{read}(k, v), \text{write}(k, v) \mid k \in \mathcal{K}, v \in \mathcal{V}\} \\
T &= (O, po), O \subseteq Op, \quad po \subseteq O \times O \\
\mathcal{H} &= (\mathcal{T}, SO, \text{level}), \quad \mathcal{T} = \{T_1, T_2, \dots, T_n\}, \\
SO &\subseteq \mathcal{T} \times \mathcal{T}, \quad \text{level} \in \{\text{SER}, \text{SI}, \text{PC}, \text{PSI}, \text{CC}, \text{RA}\} \triangleq \mathcal{L} \\
S &: \mathcal{K} \rightarrow \mathcal{V}, S(k) = \perp \text{ if } k \notin \text{dom}(S)
\end{aligned} \tag{2-2}$$

其中， $\mathcal{K}$  表示键集合， $\mathcal{V}$  表示值集合， $\perp$  表示空值； $SO$  表示会话顺序； $S$  表示快照函数。

为了后续表达异常与冲突，本文还定义事务的读集合和写集合：

$$\begin{aligned}
\text{RSet}(T) &= \{k \in \mathcal{K} \mid \exists O_i = \text{read}(k, \cdot) \in T, \forall j < i, O_j \neq \text{write}(k, \cdot)\} \\
\text{WSet}(T) &= \{k \in \mathcal{K} \mid \exists O \in T, O = \text{write}(k, v)\}
\end{aligned} \tag{2-3}$$

其中， $\text{RSet}(T)$  为事务  $T$  的读集合， $\text{WSet}(T)$  为事务  $T$  的写集合。当两个事务的写集合交集非空，即  $\text{WSet}(T_1) \cap \text{WSet}(T_2) \neq \emptyset$  时，称它们发生写冲突，记为  $T_1 \bowtie T_2$ 。

## VIS 和 AR 关系

仅用事务集合和程序顺序，还不足以描述并发事务的语义边界。真正决定隔离级别差异的，是事务之间“谁能看到谁”和“谁在逻辑上先于谁”。因此，本文在事务集合  $\mathcal{T}$  上引入两个核心关系：可见性关系 VIS 和仲裁关系 AR。

可见性关系（visibility relation） $\text{VIS} \subseteq \mathcal{T} \times \mathcal{T}$  用于描述一个事务的结果是否对另一个事务可见。若  $(T_1, T_2) \in \text{VIS}$ ，记作  $T_1 \xrightarrow{\text{VIS}} T_2$ ，则表示事务  $T_2$  的读取允许观察到事务  $T_1$  的写入结果。换言之，VIS 刻画了一个事务实际“看到了哪些历史”。

仲裁关系（Arbitration Relation） $\text{AR} \subseteq \mathcal{T} \times \mathcal{T}$  则描述全局写入顺序。若  $(T_1, T_2) \in \text{AR}$ ，记作  $T_1 \xrightarrow{\text{AR}} T_2$ ，则表示事务  $T_1$  的写入在逻辑上先于事务  $T_2$ 。

的写入。在本文中，AR 被视为事务集合上的全序关系，用于统一处理并发写入的覆盖顺序。

这两个关系并非独立存在，而满足一个关键约束：

$$\text{VIS} \subseteq \text{AR}.$$

其直观含义是：若  $T_2$  能看到  $T_1$ ，那么在全局逻辑顺序中， $T_1$  必然先于  $T_2$ 。这个约束保证了“被看见的过去”不能在全局顺序上出现在“观察者”之后，也避免了时序倒置带来的语义矛盾。可以说，VIS 描述事务实际观察到的历史片段，而 AR 为所有事务补全一个全局可比较的次序；不同隔离级别，正是通过对这两个关系施加不同强度的约束来定义的。

此外，记  $\text{VIS}^{-1}(T)$  为对事务  $T$  可见的所有事务集合，即

$$\text{VIS}^{-1}(T) = \{T' \in \mathcal{T} \mid T' \xrightarrow{\text{VIS}} T\}.$$

类似地， $\text{WriteTx}_k$  表示对键  $k$  有写操作的事务集合，用  $T \Vdash \text{op}$  表示事务  $T$  包含操作  $\text{op}$ 。

在此基础上，本文将抽象执行（abstract execution）定义为

$$\mathcal{X} = (\mathcal{T}, SO, \text{level}, \text{VIS}, \text{AR}),$$

或者简写为  $\mathcal{X} = (\mathcal{H}, \text{VIS}, \text{AR})$ 。后续所谓“某个历史满足某种隔离级别”，本质上就是在问：是否存在一组满足对应约束的 VIS 与 AR，使得该历史能够被解释为一个合法的抽象执行。

## 2.2 异常

为了在后文中更精确地讨论这些异常，下面基于 (VIS, AR) 框架，对几类典型异常进行进一步刻画。

### 因果悖逆 (Causality Violation)

因果一致性要求：若事务观察到某项更新，则也必须观察到该更新的因果前驱。如图 2-1 所示， $T_3$  观察到了  $T_2$  的评论，但没有观察到评论所依赖的  $T_1$  的帖子。这正对应前文“帖子与评论”案例中提到的异常。

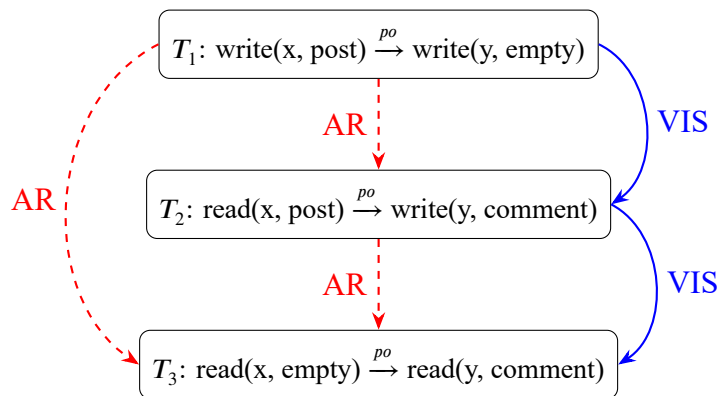


图 2-1 因果关系违背示例

### 破碎读 (Fractured Reads)

破碎读表示一个事务只看到了另一事务更新集合中的一部分，而没有看到同一事务的其余写入。如图 2-2 所示， $T_2$  读到了 Alice 关注 Bob，却没有读到 Bob 关注 Alice，说明  $T_1$  的一组相关写入没有以“整体”方式对外可见。



图 2-2 破碎读示例

### 脏读 (Dirty Read)

脏读指事务读到了另一个事务尚未提交的中间状态。如图 2-3 所示， $T_2$  在  $T_1$  提交前就读取了其写入结果。这种异常破坏了“只有提交结果才能对外生效”的事务语义。

### 更新丢失 (Lost Update)

更新丢失描述的是两个事务都基于同一个旧值做出修改，但最终只有其中一个更新在结果中保留下来。如图 2-4 所示，这类异常的根源不在于读到了错误数据，而在于两个写事务之间缺少必要的冲突协调。

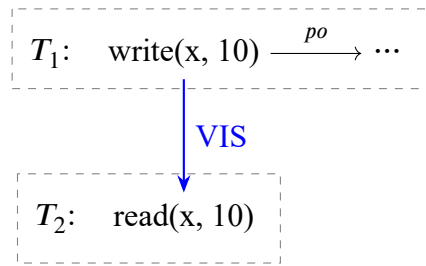


图 2-3 脏读示例

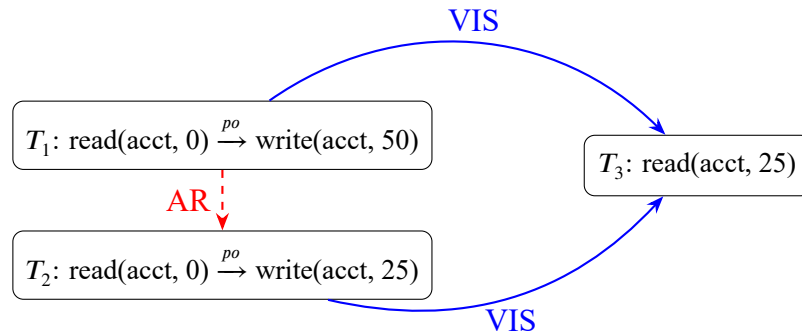


图 2-4 更新丢失示例

### 长分叉异常 (Long Fork)

长分叉指不同事务观察到了彼此不兼容的历史前缀。如图 2-5 所示， $T_3$  认为  $T_1$  在  $T_2$  之前，而  $T_4$  则观察到相反的顺序。这说明系统没有为事务提供统一的历史前缀视图。

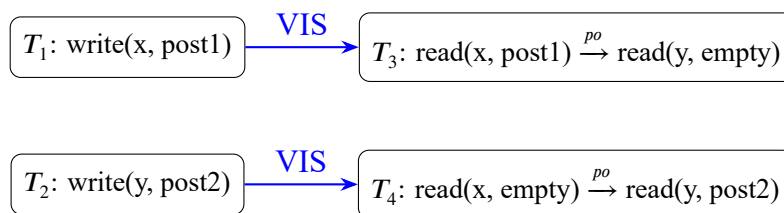


图 2-5 长分叉异常示例

### 写偏斜 (Write Skew)

写偏斜是快照类隔离级别中最典型、也最容易被忽视的异常。如图 2-6 所示，两个事务读取重叠数据，却分别更新不相交的数据项；由于二者都基于同一快照做出局部正确的判断，最终却可能共同破坏全局约束。这正对应前文“两个账户同时扣款”的案例。

$$T_1: \text{read}(\text{acct1}, 60) \xrightarrow{po} \text{read}(\text{acct2}, 60) \xrightarrow{po} \text{write}(\text{acct1}, -40)$$

$$T_2: \text{read}(\text{acct1}, 60) \xrightarrow{po} \text{read}(\text{acct2}, 60) \xrightarrow{po} \text{write}(\text{acct2}, -40)$$

图 2-6 写偏斜示例

## 2.3 公理和隔离级别

### 2.3.1 公理

前述异常描述的是“错误结果是什么样”，而公理描述的是“系统必须满足哪些结构性约束”，避免哪些异常发生。在本文采用的 (VIS, AR) 框架中，隔离级别并不是直接通过列举所有异常定义的，而是通过若干公理对合法执行进行约束；异常是否会发生，则成为这些公理的推论结果。

本文首先使用三条基础公理刻画事务最基本的可读、可见与会话语义：

- **Int (内部一致性, Internal Consistency)**: 若事务先写后读同一对象，则后续读取必须返回本事务刚写入的值。该公理保证单事务内部行为满足串行语义。
- **Ext (外部一致性, External Consistency)**: 对于事务中未被本事务覆盖的读取，其结果必须来自所有对该事务可见的写事务中、按 AR 顺序最新的那个版本。该公理保证事务只从合法的已可见历史中取值。
- **Session (会话一致性)**: 同一会话中的事务必须遵守会话顺序 *SO*。若事务  $T_1$  在会话中先于  $T_2$ ，则  $T_1$  必须对  $T_2$  可见。

在此基础上，本文再引入四条增强公理，用于约束因果传播、历史前缀、写冲突和全局可见性：

- **TransVis (传递可见性)**: 若  $T_1$  对  $T_2$  可见，且  $T_2$  对  $T_3$  可见，则  $T_1$  也必须对  $T_3$  可见。该公理用于排除因果关系违背。
- **Prefix (前缀一致性)**: 若某事务看到了  $T_2$ ，则它也必须看到 AR 顺序中位于  $T_2$  之前的全部事务。该公理保证事务观察到的是一个完整前缀，而不是带空洞的历史。

- **NoConflict** (无冲突): 若两个事务对同一对象存在写冲突, 则它们不能在彼此互不可见的情况下并发成立, 而必须通过可见性关系体现先后。
- **TotalVis** (完全可见性): 所有事务之间的可见性关系必须可比较, 即任意两个事务总有一方对另一方可见。该公理把执行效果强化为串行顺序。

以上公理的形式化表达如下:

$$\begin{aligned}
\text{Int}(T) &\equiv \forall r \in O. \forall x, v. (r = \text{read}(x, v) \wedge po_x^{-1}(r) \neq \emptyset) \\
&\Rightarrow \max_{po}(po_x^{-1}(r)) = \text{write}(x, v), \\
\text{Ext}(T) &\equiv \forall x, v. T \Vdash \text{read}(x, v) \Rightarrow \max_{AR}(\text{VIS}^{-1}(T) \cap \text{WriteTx}_x) \Vdash \text{write}(x, v), \\
\text{Session}(T) &\equiv \forall T' \in \mathcal{T}. T' \xrightarrow{\text{SO}} T \Rightarrow T' \xrightarrow{\text{VIS}} T, \\
\text{TransVis}(T) &\equiv \forall T', S \in \mathcal{T}. T' \xrightarrow{\text{VIS}} S \wedge S \xrightarrow{\text{VIS}} T \Rightarrow T' \xrightarrow{\text{VIS}} T, \\
\text{Prefix}(T) &\equiv \forall T', S \in \mathcal{T}. T' \xrightarrow{\text{AR}} S \wedge S \xrightarrow{\text{VIS}} T \Rightarrow T' \xrightarrow{\text{VIS}} T, \\
\text{NoConflict}(T) &\equiv \forall T' \in \mathcal{T}. T' \bowtie T \wedge T' \xrightarrow{\text{AR}} T \Rightarrow T' \xrightarrow{\text{VIS}} T, \\
\text{TotalVis}(T) &\equiv \forall T' \in \mathcal{T}. T' \xrightarrow{\text{AR}} T \Rightarrow T' \xrightarrow{\text{VIS}} T.
\end{aligned} \tag{2-4}$$

其中,  $po_x^{-1}(r)$  表示在程序顺序下位于读操作  $r$  之前、且作用于键  $x$  的操作集合;  $\max_{po}$  表示该集合在程序顺序中的最后一个操作;  $\max_{AR}$  表示在仲裁顺序下最近的可见写事务。

从案例角度理解, 上式中的公理与异常之间存在直接对应关系: **Ext** 排除破碎读和脏读, **TransVis** 排除因果关系违背, **NoConflict** 排除更新丢失, **Prefix** 排除长分叉, 而只有进一步强化到 **TotalVis** 时, 写偏斜等依赖全局串行效果的异常才会被排除。

### 2.3.2 隔离级别

在统一公理体系之上, 不同隔离级别可以理解为“基础公理 + 若干附加公理”的不同组合。与其把隔离级别看成一组彼此孤立的数据库术语, 不如把它们看成从弱到强逐步增加约束的过程: 越往上, 事务看到的历史越完整, 写冲突协

调越严格，最终越接近串行执行。

**读原子性 (Read Atomic, RA)**<sup>[32]</sup>仅要求满足 Int、Ext 和 Session。因此，RA 能保证事务不会读到破碎的提交结果，也不会读到未提交数据；但它并不要求因果关系可传递，也不要求冲突写入被额外协调，所以因果关系违背、更新丢失、长分叉和写偏斜仍然可能出现。

**因果一致性 (Causal Consistency, CC)**<sup>[33]</sup>在 RA 基础上加入 TransVis。因此，一旦事务读到了某个更新，它也必须继承该更新所依赖的历史。回到前文“帖子与评论”的案例，若事务看到了评论，就必须同时看到原帖，因而 CC 能排除因果关系违背；但它仍不负责处理同一对象上的并发写冲突。

**并发快照隔离 (Parallel Snapshot Isolation, PSI)**<sup>[34]</sup>在 CC 基础上进一步加入 NoConflict。这样一来，对同一对象的并发写入不能再彼此“无视对方”，而必须形成先后关系，因此 PSI 可以避免更新丢失。不过，它仍允许不同节点上的事务观察到不同的历史前缀，因此长分叉仍可能存在。

**前缀一致性 (Prefix Consistency, PC)**<sup>[35]</sup>沿着另一条路径增强约束：它在基础公理之上加入 Prefix，从而保证每个事务看到的都是全局历史的一个前缀。这样可以排除长分叉，也自然蕴含因果传播，但它并不强制禁止不同对象上的并发写入，因此仍可能发生更新丢失或写偏斜。

**快照隔离 (Snapshot Isolation, SI)**<sup>[9]</sup>可以理解为 PSI 与 PC 的结合：既要求 Prefix，保证所有事务读取某个一致快照；又要求 NoConflict，防止同一对象上的并发写冲突。因此 SI 同时排除了因果关系违背、更新丢失和长分叉，是工程实践中广泛采用的一类强隔离级别。但正如前文“两个账户同时扣款”的案例所示，SI 仍可能在跨对象约束上出现写偏斜。

**可串行化 (Serializable, SER)**<sup>[36]</sup>通过 TotalVis 把可见性关系强化为全序。此时任意两个事务都必须可比较，整个执行效果等价于某个串行顺序，因此上述异常均被排除。SER 为应用层提供最直观的编程语义，但也要求最强的协调机制，其实现代价通常最高。

表 2-1 总结了各隔离级别满足的公理组合，以及它们能否阻止典型异常。表中的 ✓ 表示该异常仍可能发生，× 表示该异常可被该隔离级别排除。

图 2-7 进一步给出了这些隔离级别在一致性约束上的偏序关系。它说明本文

讨论的六种隔离级别并不是线性罗列的概念，而是由不同公理组合形成的一组强弱关系：当一个级别蕴含另一个级别所需的全部公理时，它就在语义上更强。

表 2-1 一致性模型及其所满足的公理与异常

| $\Phi$ | Axioms                                    | Fractured<br>reads | Causality<br>violation | Lost<br>update | Long<br>fork | Write<br>skew |
|--------|-------------------------------------------|--------------------|------------------------|----------------|--------------|---------------|
| RA     | Int, Ext, Session                         | ×                  | ✓                      | ✓              | ✓            | ✓             |
| CC     | Int, Ext, Session, TransVis               | ×                  | ×                      | ✓              | ✓            | ✓             |
| PSI    | Int, Ext, Session<br>TransVis, NoConflict | ×                  | ×                      | ×              | ✓            | ✓             |
| PC     | Int, Ext, Session, Prefix                 | ×                  | ×                      | ✓              | ×            | ✓             |
| SI     | Int, Ext, Session,<br>Prefix, NoConflict  | ×                  | ×                      | ×              | ×            | ✓             |
| SER    | Int, Ext, Session, TotalVis               | ×                  | ×                      | ×              | ×            | ×             |

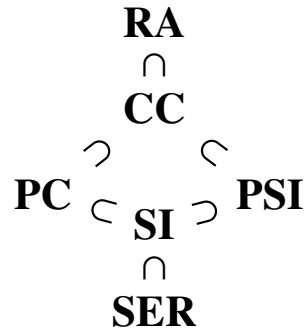


图 2-7 隔离级别间的偏序关系<sup>[18]</sup>

记  $\mathcal{X} = (\mathcal{T}, SO, \text{level}, \text{VIS}, \text{AR})$  为一个抽象执行。对任意事务  $T \in \mathcal{T}$ ，有如下包含关系：

$$\begin{aligned}
 \text{SER}(T) &\Rightarrow \text{SI}(T) \Rightarrow \text{PSI}(T) \Rightarrow \text{CC}(T) \Rightarrow \text{RA}(T), \\
 \text{SI}(T) &\Rightarrow \text{PC}(T) \Rightarrow \text{CC}(T).
 \end{aligned}
 \tag{2-5}$$

公式 2-5 给出了后文从单一隔离级别过渡到多级别组合讨论时所需要的蕴含基

础。

## 2.4 TLA<sup>+</sup> 与 TLC 基础

为了把上述定义从“可读的理论描述”进一步推进到“可验证的形式化模型”，本文在后续章节中采用 TLA<sup>+</sup> 进行规约，并使用 TLC 完成模型检查。二者的作用可以用一句话概括：TLA<sup>+</sup> 负责把系统写成精确的状态机，TLC 负责枚举这个状态机能走到的状态，并检查是否存在违背不变式的执行路径。

TLA<sup>+</sup> (Temporal Logic of Actions Plus) 是 Lamport 提出的形式化规约语言，特别适合描述并发系统和分布式系统。与实现代码不同，TLA<sup>+</sup> 不关心网络接口、线程库或存储引擎如何编写，而是直接回答两个问题：系统当前状态由哪些变量组成；系统下一步允许执行哪些动作。对于本文研究的事务系统，这种表达方式尤其合适，因为事务开始、读、写、提交、回滚，本质上都可以看成对全局状态的离散转换。

TLC 则是 TLA<sup>+</sup> 的显式状态模型检查器。给定一个有限范围内的模型配置后，TLC 会从初始状态出发，不断尝试所有满足动作定义的后继状态，并检查每个可达状态是否满足不变式，或者某条执行路径是否违反指定性质。若发现反例，TLC 会给出一条具体的状态演化轨迹，帮助定位“哪一步让系统偏离了预期语义”。

从本文问题出发，可以把二者的作用理解为一个很具体的流程：前文用 (VIS, AR) 说明“合法执行应该满足什么”，而 TLA<sup>+</sup> 则把“事务如何一步步产生这些关系”写成机器可检查的动作；TLC 再在有限状态空间中自动搜索，判断是否存在违反这些约束的执行。这样，数据库异常不再只是图中的现象，而变成了模型检查器能够自动发现和复现的反例。

图 2-8 给出了一个典型的 TLA<sup>+</sup> 规约组织方式示例。该例子虽然描述的是客户端与服务器之间的连接关系，但它很好地体现了形式化规约最基本的三个部分：初始条件 Init、状态转移关系 Next，以及用于证明安全性的状态性质。对本文的事务系统而言，也可以沿用同样的结构来描述事务开始、读、写、提交和中止等行为。

首先，Init 用来定义系统允许的初始状态。在图 2-8 中，状态由两个变量

---

```

MODULE Client_Server

CONSTANTS Server, Client
VARIABLE locked, held

Init  $\triangleq$ 
   $\wedge$  locked =  $[i \in \text{Server} \mapsto \text{TRUE}]$ 
   $\wedge$  held =  $[i \in \text{Client} \mapsto \{\}]$ 

Connect(client, server)  $\triangleq$ 
   $\wedge$  locked[server] = TRUE
   $\wedge$  held' =  $[\text{held} \text{ EXCEPT } ![client] = \text{held}[client] \cup \{server\}]$ 
   $\wedge$  locked' =  $[\text{locked} \text{ EXCEPT } ![server] = \text{FALSE}]$ 

Disconnect(client, server)  $\triangleq$ 
   $\wedge$  server  $\in$  held[client]
   $\wedge$  held' =  $[\text{held} \text{ EXCEPT } ![client] = \text{held}[client] \setminus \{server\}]$ 
   $\wedge$  locked' =  $[\text{locked} \text{ EXCEPT } ![server] = \text{TRUE}]$ 

Next  $\triangleq$ 
   $\vee \exists client \in \text{Client}, server \in \text{Server} : \text{Connect}(client, server)$ 
   $\vee \exists client \in \text{Client}, server \in \text{Server} : \text{Disconnect}(client, server)$ 

Spec  $\triangleq$  Init  $\wedge \Box[\text{Next}]_{\langle \text{locked}, \text{held} \rangle}$ 

Safe  $\triangleq$ 
   $\forall client\_i, client\_j \in \text{Client} :$ 
   $(\text{held}[client\_i] \cap \text{held}[client\_j] = \{\}) \vee (client\_i = client\_j)$ 

```

---

图 2-8 Client\_Server 规约

locked 和 held 组成：前者记录每个服务器当前是否空闲，后者记录每个客户端当前持有哪些服务器。Init 要求所有服务器一开始都处于可用状态，同时所有客户端的持有集合都为空。也就是说，TLA<sup>+</sup> 中的“状态”本质上就是对一组状态变量的整体赋值；而对事务系统来说，类似的状态变量可以包括活动事务集合、每个事务的读写集、版本存储、可见关系和仲裁顺序等，它们共同刻画了某一时刻系统的全貌。

其次, Next 用来定义一步状态转移是如何发生的。图中的 Connect(client, server) 和 Disconnect(client, server) 都是具体动作：它们一方面给出动作发生前必须满足的前置条件，例如 Connect 只能选择当前尚未被占用的服务器，Disconnect 只能释放该客户端已经持有的服务器；另一方面通过带撇号的变量，如 held' 和 locked'，给出执行该动作后的新状态。这里的撇号表示“下一状态中的变量值”，因此一个动作实际上就是从当前状态到下一状态的约束关系。Next 再把所有可能的一步动作取析取，表示系统下一步可以执行任意一个被允许的动作。对事务系统而言，这种编码方式同样适用：开始

事务、读、写、提交和回滚都可以写成类似的状态转移谓词，而整个系统行为就是这些动作反复交替形成的状态序列。

在  $\text{Init}$  与  $\text{Next}$  之上，规约通常进一步写成

$\text{Spec} = \text{Init} / [][\text{Next}]_{\langle \text{vars} \rangle}$ 。其含义是：系统行为必须从某个满足  $\text{Init}$  的初始状态出发，此后每一步都满足  $\text{Next}$  所允许的转移关系，或者保持变量不变。换言之，规约并不是单独描述某一个状态，而是在定义一条完整执行轨迹应当满足什么条件。TLC 模型检查时，正是从  $\text{Init}$  出发，不断枚举满足  $\text{Next}$  的后继状态，从而构造所有可达行为，再检查这些行为是否违反目标性质。

图中给出的  $\text{Safe}$  则是一个状态性质，用来表达“不会有两个不同客户端同时持有同一台服务器”。在实际证明中，往往还会引入一个更强的归纳不变式  $\text{Ind}$ ：它不仅蕴含我们真正关心的安全性质  $\text{Safe}$ ，还满足两个条件，即  $\text{Init} \Rightarrow \text{Ind}$ ，以及任意一步转移都保持  $\text{Ind}$  成立。只要这两个条件成立，就可以推出在所有可达状态上  $\text{Ind}$  都为真，进而得到  $\text{Ind} \Rightarrow \text{Safe}$ ，于是规约满足  $\text{Safe}$ 。因此，从  $\text{TLA}^+$  的角度看，规约正确性通常定义为：由  $\text{Spec}$  生成的所有可达行为都满足给定的不变式或时序性质；而对本文的事务系统来说，这就对应于所有由操作语义产生的执行都不会违反可见性约束、仲裁顺序约束以及各隔离级别要求的一致性公理。

可以用一个简单例子理解“动作”的含义。若事务  $T_1$  执行写操作“ $\text{write}(x,1)$ ”，那么动作本身并不是一句程序语句，而是一个状态约束：执行前， $T_1$  处于活动状态；执行后， $T_1$  的写集合中新增键值对  $(x,1)$ 。若随后另一个事务  $T_2$  执行读操作“ $\text{read}(x,1)$ ”，则该动作是否允许发生，取决于当前状态下哪些事务对  $T_2$  可见、以及  $x$  在可见历史中的最新版本是什么。正是这种“以前置条件和后置状态”来描述行为的方式，使  $\text{TLA}^+$  能够天然地表达事务系统的语义，而不必依赖具体编程语言实现。

不同于面向实现的程序代码， $\text{TLA}^+$  规约会主动省略与语义无关的工程细节，例如网络报文格式、线程调度机制和缓存实现方式，而保留那些真正影响正确性的状态变量和动作约束。这样做的目的并不是弱化模型，而是把验证重点聚焦到事务可见性、提交顺序和隔离约束本身，从而提高模型的可验证性与可解释性。

在本文中， $\text{TLA}^+$  的作用是把混合隔离级别下的事务执行过程写成精确的形

式化规约，使事务状态、读写行为、提交顺序以及由此诱导出的 VIS 和 AR 关系都能被统一地表达出来；TLC 的作用则是在给定有限模型参数后，对这些规约进行自动枚举和检查，验证系统是否始终满足相应隔离级别的一致性公理，并在性质不成立时给出可复现的反例执行轨迹。二者结合起来，使本文不仅能够从理论上定义“什么是合法执行”，还能够进一步验证“所设计的操作语义是否确实只能产生合法执行”。

## 2.5 本章小结

本章在保留既有形式化素材的基础上，对第二章内容进行了以案例为主线的重组。首先，借助帖子评论、更新丢失和写偏斜等案例说明数据库异常如何引出隔离需求，并进一步说明分布式环境下开展数据库建模的必要性。其次，围绕事务、快照、执行历史、VIS 与 AR 给出统一的数据库建模方式，为后续异常、公理和隔离级别的讨论提供共同语义基础。随后，通过典型异常、公理体系、表 2-1 和图 2-7，对 RA、CC、PSI、PC、SI 和 SER 的约束关系进行了统一表达。最后，介绍了  $TLA^+$  与 TLC 的基本作用，并给出混合隔离级别抽象执行的判定基础。上述内容为后续章节中的操作语义设计、 $TLA^+$  建模和模型检查实验提供了直接的理论支撑。



### 第三章 面向混合隔离级别的系统操作语义设计

前文已经从事务异常、隔离级别与一致性语义三个层面建立了本文的理论基础：一方面，借助因果关系违背、更新丢失和写偏斜等典型异常，说明了不同隔离级别在可见性与顺序约束上的本质差异；另一方面，基于抽象执行模型，对事务历史、快照、VIS 与 AR 关系以及相应一致性公理进行了统一定义。由此，前文已经回答了“系统应当满足什么语义约束”的问题。

在此基础上，本章进一步关注“这些语义约束如何在系统中被实现”这一核心问题。具体而言，本章将面向支持混合隔离级别的分布式事务系统，给出一套与前述抽象执行语义相对应的操作语义设计，说明系统状态如何组织、事务操作如何驱动状态演化，以及不同隔离级别所要求的可见性、定序与冲突约束如何落实到具体协议机制中。通过这一过程，本文希望在抽象一致性定义与系统实现机制之间建立一条清晰且可验证的映射关系，为后续的  $TLA^+$  建模、模型检查以及原型实现提供语义基础。

#### 3.1 混合隔离级别问题

前文讨论的 RA、CC、PSI、PC、SI 和 SER，默认都以“同一批事务遵循同一隔离级别”为背景进行定义。但对本文关注的系统而言，关键问题并不在于如何分别实现某一个隔离级别，而在于如何使语义需求不同的事务在同一执行历史中同时成立。所谓**混合隔离级别**，是指允许不同事务根据自身约束和性能需求采用不同的隔离级别，并要求它们在同一个抽象执行框架下同时满足各自语义。

从系统需求看，引入混合隔离级别并不是为了增加理论复杂度，而是为了更贴近真实工作负载<sup>[37]</sup>。以电商、社交平台或分布式键值数据库为例，并不是所有事务都具有同等重要性：商品浏览、点赞计数、普通日志写入等操作往往更强调响应时间与系统吞吐；而库存扣减、账户转账、元数据变更或配置发布等操作，则必须满足更严格的一致性要求。若对所有事务一律采用可串行化隔离，虽

然语义最直观，但系统需要为大量实际上并不敏感的事务支付额外协调成本；反之，若统一退化到较弱隔离，又会使关键事务暴露在不可接受的异常风险之下。因此，让不同事务按需选择隔离级别，既是性能优化手段，也是本文系统设计需要正面回答的语义问题。

但也正因为不同事务共存于同一系统中，混合隔离级别比单一隔离级别更难实现。单一隔离级别场景下，所有事务遵守同一套约束，系统只需判断整体是否满足该级别；而混合隔离级别场景下，系统既要允许较弱事务保留更多并发，又要保证较强事务的语义不被前者破坏。换言之，问题不再是“系统是否满足某个隔离级别”，而是“系统能否同时容纳多种强度不同、但彼此相互影响的隔离要求”。这也是本文需要在第三章从系统视角重新引入该问题的原因。

形式化地说，对一个抽象执行

$$\mathcal{X} = (\mathcal{T}, SO, \text{level}, \text{VIS}, \text{AR}),$$

其中  $\text{level}(T)$  不再对所有事务取同一个值，而是允许不同事务分别标注为 RA、CC、PSI、PC、SI 或 SER。若对任意事务  $T \in \mathcal{T}$ ，其隔离级别  $\text{level}(T)$  所要求的全部公理均被满足，则称  $\mathcal{X}$  在该混合隔离级别配置下成立。这里的关键不是把不同级别简单并列摆放，而是要求存在同一组 VIS 与 AR，能够同时解释所有事务所观察到的历史。

若一个历史  $\mathcal{H}_i$  在相应混合隔离级别配置下成立，则必然存在某个抽象执行  $\mathcal{X} = (\mathcal{H}_i, \text{VIS}, \text{AR})$  与之对应且满足全部约束。对于统一隔离级别场景，这一判定会退化为常见的单级别情况；而在混合隔离级别场景下，若历史  $\mathcal{H}$  同时包含 SI 与 SER 两类事务，且在给定的 level 配置下存在合法抽象执行，则可记作  $\mathcal{H} \models \text{SI-SER}$ 。类似记号也可用于描述其他隔离级别组合。

因此，混合隔离级别的核心问题并不是“每个事务各自满足什么”，而是“这些事务能否在同一个全局历史解释下同时满足各自约束”。一个事务单独看可能满足自身级别，但只要它所依赖的可见历史与其他事务要求的约束发生冲突，整个执行仍然可能不成立。对本文而言，这一判断不再停留在理论定义层面，而需要继续向下落实为系统中的状态组织、时间戳分配、读写规则与提交认证机制。

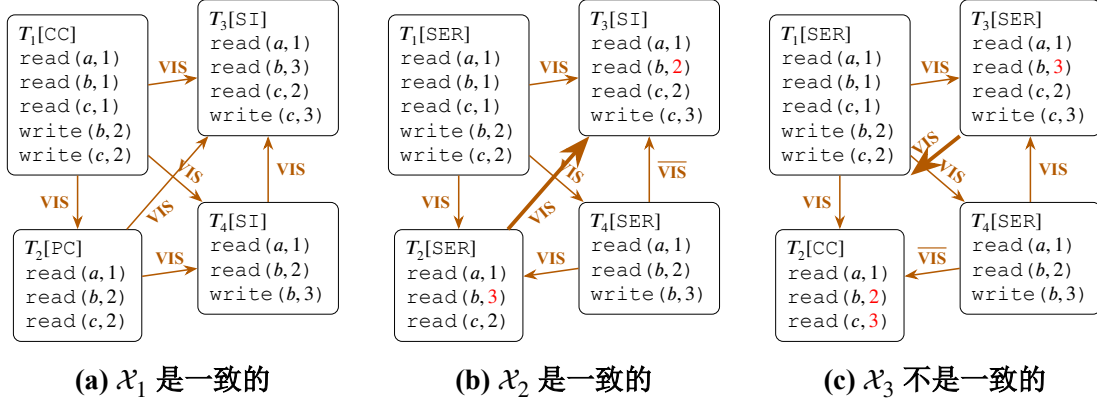


图 3-1 混合隔离级别下抽象执行判定示例

### 混合隔离级别下的抽象执行判定

图 3-1 给出了三个混合隔离级别的抽象执行示例。子图 (a) 中， $\mathcal{X}_1$  同时包含 CC、PC 与 SI 三类事务，但它们观察到的历史可以由同一组 VIS 与 AR 关系统一解释，因此该执行是成立的。子图 (b) 中， $\mathcal{X}_2$  同时包含 SER 与 SI 两类事务，虽然隔离要求不同，但仍未出现彼此冲突的可见性约束，因此该执行同样成立。

子图 (c) 展示了更关键的反例： $\mathcal{X}_3$  同时包含 SER 与 CC 事务，其中 CC 事务  $T_2$  读到了  $b = 2$  和  $c = 3$ ，这意味着它的可见历史无法与 SER 事务要求的全局可比较顺序同时成立。也就是说，单独看  $T_2$ ，它似乎没有超出 CC 的允许范围；但把它放入整个混合执行中，就会破坏其他事务所需的更强约束，因此  $\mathcal{X}_3$  不是一致的。

这张图反映了混合隔离级别的一个关键特征：它并不意味着“每个事务只对自己的隔离级别负责”，而是要求整个执行存在一个统一的全局解释。若某个事务的可见历史与其他事务所需的约束发生冲突，即使它单独看来满足自身级别，也可能导致整个混合执行不成立。因此，混合隔离级别的正确性判定应理解为全局判定，而不是若干局部判定结果的简单叠加。对后续系统设计而言，这意味着当系统允许不同事务选择不同隔离级别时，协议设计需要显式考虑它们之间的相互影响，尤其要避免较弱事务暴露出会破坏较强事务语义的可见历史。

## 3.2 系统架构建模

我们采用一个经典的分布式数据库的系统模型，遵循客户端路由架构，这种架构被如 MongoDB 分布式数据库广泛使用。MongoDB 采用分片集群（Sharded

Cluster) 架构, 主要包含三个核心组件: 分片服务器 (Shard)、配置服务器 (Config Server) 和路由服务器 (mongos)。分片服务器负责实际数据的存储; 配置服务器存储集群的元数据和分片配置; 路由服务器作为客户端的接入点, 负责将请求路由到正确的分片上。这种架构具有卓越的水平扩展能力和高可用性。

选择该架构来实现混合隔离级别数据库原型具有以下合理性<sup>[38]</sup>: 首先, 客户端路由架构实现了事务协调与数据存储的解耦, 便于我们在路由层或客户端灵活实现不同的事务协调协议以支持多种隔离级别; 其次, 其基于混合逻辑时钟 (HLC) 的时间戳机制为分布式事务的定序提供了天然支持, 契合本文的形式化模型; 最后, 分片架构使得系统能够通过增加节点线性提升吞吐量, 能够验证混合隔离级别在大规模分布式环境下的性能表现。

客户端主要负责接受用户请求、维护与路由服务器的连接, 并将同一事务的全部操作稳定地发送到同一个路由节点。这里的关键不是客户端本身执行复杂的并发控制, 而是保证事务亲和性: 事务一旦开始, 后续读、写、提交和中止都复用同一路由节点, 从而使事务的全局元数据、隔离级别和参与分片集合能够被一致地维护。与此同时, 客户端后台周期性地从配置服务器拉取可用路由节点集合, 并在开启新事务时通过负载均衡策略选择入口节点。这一设计把“入口选择”和“事务执行”分离开来, 避免了事务执行过程中发生无约束的路由漂移。

配置服务器存储集群元数据, 包括路由节点列表、分片键空间划分及节点健康状态。它一方面接收路由节点和分片节点的注册信息, 另一方面通过心跳或租约机制判断节点是否仍然可用, 并为客户端和路由节点提供最新的分片映射。对本文而言, 配置服务器的作用并不在于参与事务语义本身, 而在于为后续所有操作提供一个稳定、可查询的全局拓扑视图, 使得键到分片的映射在事务执行期间是可解释的。

路由服务器是本章操作语义的协调核心。每个活动事务在路由层都维护一份全局控制信息, 至少包括事务标识、隔离级别、开始时间戳以及当前已经访问过的分片集合。当客户端发起读或写操作时, 路由节点首先根据键查找目标分片, 然后将请求连同事务上下文转发给对应分片; 若该分片此前尚未参与该事务, 则将其加入事务的参与分片集合。事务提交时, 路由节点依据参与分片数选择提交路径: 若事务仅访问一个分片, 则可直接走单分片提交路径; 若涉及多个分片, 则由路由节点充当协调者, 按准备与决议两个阶段组织提交。由此, 事务

的全局组织与本地执行被清晰地划分在不同层面。

分片服务器负责实际的数据存储、版本管理和局部并发控制。每个分片维护一个基于 MVCC 的版本存储<sup>[39]</sup>，并为活动事务按需建立局部缓冲区，用于保存事务在该分片上的读集合、写集合以及必要的辅助元数据。读操作遵循“先读本地写集，再读已提交版本”的顺序：如果事务已经在本地写缓冲区中修改过某个键，则后续读操作直接返回缓冲区中的最新值；否则，系统根据事务快照时间戳从 MVCC 存储中选择一个满足可见性约束的已提交版本。写操作并不立即覆盖共享存储，而是先写入事务的本地缓冲区，待提交阶段统一验证并发布。这样的设计同时支撑了事务内部一致性和多版本快照读取。

从体系结构与语义分工的角度看，客户端、路由节点和分片节点分别承担了三个不同层次的职责。客户端负责保持会话连续性和事务亲和性，使同一事务的上下文不会因入口漂移而被破坏；路由节点负责维护事务的全局控制信息，并把一次逻辑事务组织成若干面向具体分片的局部操作；分片节点则真正决定某个读是否可见、某个写是否能够被发布以及某次提交是否需要被中止。这样的分工使得系统既能够在全局层面统一处理时间戳、参与分片集合和提交路径，又能够在局部层面执行与数据项和版本链紧密相关的并发控制逻辑。对于混合隔离级别场景而言，这一点尤为重要，因为不同隔离级别的差异并不主要体现在客户端接口上，而是体现在事务在路由层如何获得快照、在分片层如何解释可见版本、以及在提交阶段如何接受不同强度的认证。

### 事务状态与关键元数据

为了把上述组件行为统一为可推理的操作语义，系统还需要为事务维护一组稳定的关键元数据。首先是**开始时间戳** *sts*，它决定了事务读取历史时所依据的初始快照边界；其次是**提交时间戳** *cts*，它在事务成功提交后用于确定该事务在全局仲裁序中的位置。对于因果相关的隔离级别，客户端还会维护**依赖时间戳** *dep*，用来刻画会话内已经建立的因果先后关系；对于 SER 事务，系统还需要维护**检查集** (checkset)，以记录提交前必须重新验证的读写对象集合。

除时间相关元数据外，事务在执行过程中还会逐步形成若干集合结构。其一是**参与分片集合**，由路由节点维护，用于记录当前事务已经访问过哪些分片，并据此决定最终采用单分片提交还是跨分片两阶段提交；其二是**读集合与写集合**，

由分片节点维护，用于分别记录事务已经读取过哪些键以及尚未发布但准备提交的更新内容；其三是**准备状态**，用于描述事务在两阶段提交中是否已经通过局部验证并获得预分配时间戳。对操作语义而言，这些元数据不仅是实现细节，也直接参与后续对 VIS、AR 以及各类一致性公理的判定。

从语义角度看，这些元数据承担的是“把一次运行中的事务解释为抽象执行元素”的桥梁作用。*sts* 决定事务可见历史的上界，*cts* 决定事务在仲裁序中的位置，*dep* 决定会话依赖能否被后继事务继承，而读写集与检查集则分别决定事务在提交时需要接受哪一种冲突认证。因此，第三章虽然讨论的是系统协议，但其中真正被设计和约束的对象，始终是这些与抽象执行一一对应的状态量。也正因如此，后续第四章才能将本章的系统行为进一步翻译为  $TLA^+$  中的状态变量与状态转移。

从执行流程上看，本文的事务操作语义可以概括为四个相互衔接的阶段。第一，**事务启动阶段**：客户端指定事务隔离级别后，路由节点为事务建立全局上下文，并根据隔离级别确定初始快照时间戳的来源。第二，**读写执行阶段**：读操作根据事务上下文选择可见版本，写操作进入局部缓冲区，事务在此阶段逐步形成读写集与依赖信息。第三，**提交验证阶段**：系统根据隔离级别要求，对事务的读写集、检查集、依赖时间戳或准备状态进行验证，以判断当前执行是否仍然能够被解释为满足目标公理的抽象执行。第四，**结果发布阶段**：若验证通过，则为事务分配提交时间戳并将写集写入版本存储；若验证失败，则丢弃缓冲区并中止事务。与传统面向实现的伪代码相比，这种分阶段描述更能突出本文关注的语义主线，即一个事务如何从“获得快照”走向“形成历史”，并最终影响 VIS 与 AR 的构造。

在跨分片事务处理中，系统仍需依赖两阶段提交 (2PC)<sup>[40]</sup> 来保证原子性。路由节点在准备阶段向所有参与分片发起验证请求；任一分片报告冲突或验证失败，事务即整体中止；只有当全部分片均通过验证后，协调者才进入提交阶段并通知各分片发布写集。这里需要注意的是，两阶段提交在本文中的作用主要是保证多分片更新的原子发布，而真正决定某个事务是否满足目标隔离级别的，仍然是各分片在准备阶段执行的快照验证、依赖检查和冲突检测逻辑。

如果把上述跨分片控制路径进一步写成协议骨架，则可得到算法 1。它保留了本章最关键的控制点：事务启动时建立全局上下文、提交时依据参与分片数在

QuickCommit 与 2PC 之间切换，以及中止时统一清理全局事务状态。

---

**Algorithm 1** 路由服务器协议：开始、提交与回滚
 

---

**Require:** 事务全局信息 txnInfo。

```

1: procedure STARTTXN(txn_id, isolation_level)
2:   start_ts  $\leftarrow$  hlc.Now()  $\triangleright$  分配事务开始时间戳
3:   txnInfo[txn_id]  $\leftarrow$  {start_ts, isolation_level, shards :  $\emptyset$ }
4:   return SUCCESS

5: procedure COMMIT(txn_id)
6:   if  $\neg$ txnInfo.contains(txn_id) then
7:     return NOTTRANSACTION
8:   shards  $\leftarrow$  txnInfo[txn_id].shards
9:   if |shards| = 1 then
10:    result  $\leftarrow$  shard.quickCommit(txn_id)  $\triangleright$  单分片快速提交
11:  else
12:    result  $\leftarrow$  TwoPhaseCommit(txn_id, shards)
13:  txnInfo.remove(txn_id)  $\triangleright$  清理事务信息
14:  return result

15: procedure TWOPHASECOMMIT(txn_id, shards)  $\triangleright$  跨分片两阶段提交
16:   for shard  $\in$  shards do  $\triangleright$  准备阶段
17:     if shard.prepare(txn_id)  $\neq$  SUCCESS then
18:       for s  $\in$  shards do
19:         s.abort(txn_id)  $\triangleright$  回滚所有分片
20:       return CONFLICT
21:   for shard  $\in$  shards do  $\triangleright$  提交阶段
22:     shard.commit(txn_id)
23:   return SUCCESS

24: procedure ABORT(txn_id)
25:   if txnInfo.contains(txn_id) then
26:     shards  $\leftarrow$  txnInfo[txn_id].shards
27:     for shard  $\in$  shards do  $\triangleright$  通知所有分片中止
28:       shard.abort(txn_id)
29:     txnInfo.remove(txn_id)
30:   return SUCCESS
  
```

---

针对单分片事务这一常见场景，本文引入了 QuickCommit 优化。其基本思想是：若事务自始至终只访问一个分片，则可以将“准备”和“提交”折叠为一次局部决策，在同一个分片上完成时间戳分配、冲突验证和写集发布。这样做不会改变语义判定条件，因为事务仍需接受与常规提交路径一致的可见性与冲突检查；它所节省的只是跨节点协调所引入的额外网络往返和中间状态维护开销。因此，QuickCommit 可以被理解为一种在不削弱语义约束前提下对提交流程进行的工程化优化，而不是对隔离语义本身的放宽。

为了更清楚地说明这种折叠优化没有改变验证条件，算法 2 给出了单分片快

速提交的局部流程。可以看到，它仍然保留了基于读集合的冲突检测与基于提交时间戳的版本发布，只是将原本分离的 Prepare 与 Commit 两步合并为一次本地决策。

---

**Algorithm 2 QuickCommit 快速提交优化**


---

**Require:** 事务缓存信息 buff, MVCC 存储引擎 storage, 混合逻辑时钟 hlc。

```

1: procedure QUICKCOMMIT(txn_id) ▷ 分片服务器快速提交
2:   if ¬buff.contains(txn_id) then
3:     return SUCCESS ▷ 无操作则直接成功
4:   commit_ts ← hlc.Now() ▷ 获取提交时间戳
5:   readSet ← buff[txn_id].readSet
6:   for (key, _, read_ts) ∈ readSet do ▷ 冲突检测
7:     if storage.hasNewerVersion(key, read_ts, commit_ts) then
8:       Abort(txn_id)
9:     return CONFLICT
10:  writeSet ← buff[txn_id].writeSet
11:  for (key, value) ∈ writeSet do ▷ 持久化写集
12:    storage.write(key, value, commit_ts)
13:  buff.remove(txn_id) ▷ 清理事务缓存
14:  return SUCCESS

```

---

需要强调的是，本节虽然减少了逐接口展开的伪代码描述，但并不意味着系统行为被“抽象掉”了。相反，上述叙述把注意力从实现细节转移到更适合论文表达的语义要点上：事务何时获得快照、读取时依据什么规则选择版本、提交时依据什么条件决定成功或中止，以及这些条件如何随隔离级别增强而逐步收紧。对于学位论文而言，这种表述方式更有利于将操作语义与前文的一致性公理以及后文的形式化验证一一对应起来。

### 3.3 混合逻辑时钟

在分布式系统中，时钟（Clock）是用于刻画事件发生顺序、界定事件间时序关系的核心抽象工具，其本质是为系统中的每一个事件分配一个可比较的标识（时间戳，Timestamp），通过时间戳的序关系映射事件的发生序，为分布式一致性、事件追踪与事务排序提供基础支撑。

时钟的意义是区分事件发生的先后顺序，刻画事件间的因果关系，从而为分布式系统中的一致性协议、并发控制、故障恢复等机制提供时间基础。对于事件  $e_1$  和  $e_2$ ，如果  $e_1$  发生在  $e_2$  之前，我们希望通过时钟分配的时间戳  $TS(e_1)$  和  $TS(e_2)$  满足  $TS(e_1) < TS(e_2)$ 。这种时间戳的序关系使得系统能够判断事件的先后

顺序，进而实现一致性保证和正确的并发控制。

从理论与工程分类来看，时钟主要分为两类：一类是物理时钟（Physical Clock），基于现实世界的物理时间（如 UTC 时间），通过网络时间协议（NTP）、精确时间协议（PTP）等实现分布式节点间的时钟同步，为事件分配具有人类可读语义的时间戳，但受网络延迟、时钟漂移等因素影响，节点间时钟无法实现完全同步，存在固有误差且可能出现时钟回拨；另一类是逻辑时钟（Logical Clock），不依赖物理时间，仅通过节点本地计数或消息交互更新时间戳，核心目标是捕获事件间的因果依赖关系，确保具有因果关联的事件拥有可比较的时间戳，但缺乏物理时间语义，无法支撑基于物理时间的系统需求。

混合逻辑时钟（Hybrid Logical Clock, HLC）<sup>[41]</sup>结合了物理时钟和逻辑时钟的优点，能够在分布式系统中提供一个全局一致的时间戳。HLC 由物理时间和逻辑计数器组成，严格遵循时钟条件，有效弥补了纯物理时钟与纯逻辑时钟的固有缺陷，能够有效地处理网络延迟和节点故障带来的时间不一致问题。

一个混合逻辑时钟时间戳通常表示为一个二元组  $\langle pt, lc \rangle$ ，其中  $pt$  是物理时间戳， $lc$  是逻辑计数器，有：

$$\langle pt_1, lc_1 \rangle < \langle pt_2, lc_2 \rangle \iff (pt_1 < pt_2) \vee (pt_1 = pt_2 \wedge lc_1 < lc_2) \quad (3-1)$$

它向外提供两个典型接口 Now 和 Update，如算法 3 所示：

Now()会读取当前系统时间。如果系统时间大于上次的时间戳，则更新物理时间并将逻辑计数器重置为 0；如果系统时间等于或小于上次时间戳，则递增逻辑计数器，用于给本地事务调用；Update将本地时钟与远程时间戳同步。它会取本地时间、远程时间和当前物理时间三者中的最大值，并相应地调整逻辑计数器。这是保证因果关系（Happens-before）的关键，当接收到带有远程 HLC 时间戳的消息时调用。HLC 的这两个接口，在本文设计分布式数据库混合隔离级别下的事务操作语义时至关重要。

### 3.4 面向混合隔离级别的操作语义

在上一章，我们提到了各个隔离级别在需要满足的公理，以集合的视角看，它们存在着包含关系，如表 3-1 所示。对于不同隔离级别的事务，我们需要在系

**Algorithm 3** 混合逻辑时钟管理

**Require:** 全局变量 (wall, logical)  $\triangleright$  本地时钟状态

```
1: procedure Now(pt)  $\triangleright$  生成本地时间戳
2:   if pt > wall then
3:     wall  $\leftarrow$  pt
4:     logical  $\leftarrow$  0
5:   else
6:     logical  $\leftarrow$  logical + 1
7:   return (wall, logical)

8: procedure UPDATE(msg, pt)  $\triangleright$  同步远程时间戳
9:   old_wall  $\leftarrow$  wall
10:  wall  $\leftarrow$  max(old_wall, msg.wall, pt)
11:  if wall = old_wall  $\wedge$  wall = msg.wall then
12:    logical  $\leftarrow$  max(logical, msg.logical) + 1
13:  else if wall = old_wall then
14:    logical  $\leftarrow$  logical + 1
15:  else if wall = msg.wall then
16:    logical  $\leftarrow$  msg.logical + 1
17:  else
18:    logical  $\leftarrow$  0
```

统中配置不同的组件以实现相应的公理组合，从而满足隔离级别的语义要求。更高一致性的隔离级别是低一致性隔离级别的子集，在实现阶段，可以通过对操作语义的某些过程中添加一些新的限制以实现新的公理，从而满足更高隔离级别的要求。

**表 3-1 隔离级别及其公理组合**

| 隔离级别                      | 强制满足的公理组合                                            | 核心语义说明                 |
|---------------------------|------------------------------------------------------|------------------------|
| <b>RA (Read Atomic)</b>   | $\text{Int} \wedge \text{Ext} \wedge \text{Session}$ | 保证原子可见性，防止断裂读。         |
| <b>CC (Causal)</b>        | $\text{RA} \wedge \text{TransVis}$                   | 保证因果一致性。               |
| <b>PC (Prefix)</b>        | $\text{RA} \wedge \text{Prefix}$                     | 保证观察到的是全局序的一致前缀。       |
| <b>PSI (Parallel SI)</b>  | $\text{CC} \wedge \text{NoConflict}$                 | 因果一致性 + 禁止丢失更新。        |
| <b>SI (Snapshot)</b>      | $\text{PC} \wedge \text{NoConflict}$                 | 经典快照隔离：一致性快照 + 禁止丢失更新。 |
| <b>SER (Serializable)</b> | $\text{SI} \wedge \text{TotalVis}$                   | 可串行化：所有前驱必须可见。         |

实现各个隔离级别，本质上是以统一事务流程为骨架，在不同阶段施加不同强度的约束。所有事务都共享相同的基本执行框架：启动时确定快照时间，执行期间维护局部读写信息，提交时完成验证与写集发布；而隔离级别之间的差异，则体现在快照如何选择、执行期间需要追踪哪些依赖、读请求是否需要等待、以及提交阶段需要检查哪些冲突。

## 各隔离级别的语义定位

为了避免不同隔离级别仅以表格形式出现而缺乏纵向展开，下面对 CC、PC、PSI、SI 和 SER 在本文系统中的语义定位分别作简要说明。

**CC (Causal Consistency)** 关注的是因果相关事务之间的先后关系不被破坏。对 CC 事务而言，系统并不要求所有事务共享同一个全局快照，而是要求当前事务至少能够观察到其因果依赖链上的所有前驱。因此，CC 的关键不在于读取某个“统一时刻”的数据库状态，而在于通过依赖时间戳把先前会话中已经建立的因果顺序持续传递给后继事务。相较于更强的前缀型语义，CC 允许更高的并发自由度，但也意味着不同事务观察到的全局历史不必具有统一前缀结构。

**PC (Prefix Consistency)** 则将重点从因果传播转移到全局顺序的一致前缀。PC 不要求显式维护每条因果依赖，而是要求事务所见历史必须能够解释为某个全局仲裁序的前缀。因此，在系统实现中，PC 的关键在于快照时间的选取以及读取阶段对未决前驱事务的等待机制。通过这种方式，事务虽然可能看不到所有并发事务，但它所看到的提交历史不会出现“跳过前驱、直接观察后继”的空洞现象。与 CC 相比，PC 对全局观察结果施加了更强约束，但不追求因果依赖的精细传播。

**PSI (Parallel Snapshot Isolation)** 可以看作在 CC 基础上加入 NoConflict 约束后的结果。它既保留了因果一致性场景下依赖传播的能力，又进一步要求并发写冲突不能同时成功提交。换言之，PSI 不仅关心“该看到的因果前驱是否可见”，也关心“是否存在会造成丢失更新的并发写入”。在系统层面，这意味着 PSI 事务既需要继承依赖时间戳，又需要在提交阶段接受写集认证。因此，PSI 在语义上位于“因果有序”与“写冲突受控”之间，是一种兼顾依赖传播与版本安全的混合机制。

**SI (Snapshot Isolation)** 则更强调一致快照与写冲突禁止的结合。SI 事务在启动时获取一个统一快照，后续所有读操作都基于该快照解释版本可见性；同时，在提交时还必须通过写冲突认证，避免两个并发事务在彼此不可见的情况下同时覆盖同一数据项。与 PSI 不同，SI 并不以客户端依赖传播为基础，而是依赖全局快照时间和前缀一致读取来确保事务观察到的历史是自洽的。因此，SI 的核心特点可以概括为：读时读取统一快照，写时禁止冲突覆盖。

**SER (Serializable)** 是本文讨论的最强隔离级别。它不仅要求事务读取某个自洽快照，也不仅要求并发写冲突被禁止，还要求事务的整个执行结果能够被解释为某个串行顺序中的一次合法执行。正因如此，SER 不能仅依赖写集认证，而必须把读集合也纳入提交前验证。系统需要进一步判断：事务在执行期间所依据的读结果，是否会因为并发事务在其后提交而失去串行化解释。SER 因而对应着最高的验证强度，也对应着最大的控制开销；但作为回报，它能够排除写偏斜等更弱隔离级别仍可能容忍的异常。

### 统一事务执行流程

对任意事务  $T$ ，本文采用如下统一的操作语义框架：

1. **启动**：路由节点根据事务隔离级别为其分配初始上下文，包括快照时间戳  $sts$ 、隔离级别标记以及空的参与分片集合。
2. **读**：事务首先检查本地写缓冲区；若命中，则直接返回本事务尚未提交的最新值；否则根据  $sts$  和当前隔离级别对应的可见性规则，从分片上的 MVCC 版本链中选择可见版本，并将读信息记录到本地读集合或检查集中。
3. **写**：事务把写入暂存到局部写缓冲区，而不是立即发布到共享存储；因此，执行阶段形成的是“暂存的候选更新”，而非已提交历史的一部分。
4. **提交**：系统在准备阶段检查当前事务是否仍与其快照和依赖信息相容。验证通过后，事务获得提交时间戳  $cts$  并发布写集；验证失败则中止。提交成功后， $cts$  参与定义抽象执行中的仲裁顺序，而事务的读结果与依赖关系共同决定其可见性关系。

在这一框架下，事务内部一致性 (Int) 由“先查本地写集，再查共享版本”的读取规则保证，因此事务始终能够观察到自己的写入；外部一致性 (Ext) 则由快照读取规则保证，即事务只能读到不晚于其快照时间并满足相应隔离约束的已提交版本。基于这两个基础性质，系统再进一步叠加因果依赖、前缀顺序、写冲突禁止以及全前驱可见等更强约束，从而得到更高强度的隔离级别。

### 公理到机制的映射

在上述统一流程基础上，我们继续讨论 Session、TransVis、Prefix、NoConflict

和 TotalVis 等公理如何落入具体机制。

对于 Session 和 TransVis 公理，其核心在于维护因果一致性。系统通过依赖追踪（Dependency Tracking）机制来实现这一目标。具体而言，客户端会维护一个依赖时间戳  $dep$ ，该时间戳是混合逻辑时钟（HLC）的一个实例。每当客户端成功提交一个事务或者读取到数据时，都会使用由于该操作产生的提交时间戳  $commit\_ts$  来更新本地的依赖时间戳，即  $dep \leftarrow \max(dep, commit\_ts)$ 。当一个新的因果一致性（CC）或并行快照隔离（PSI）级别的事务  $T$  启动时，系统将其快照时间戳  $T.sts$  设置为当前的  $dep$ 。这一机制确保了当前事务的快照时间必定大于所有与其存在因果依赖关系的事务的提交时间，从而严格保证了因果相关的事务对当前事务可见。

对于 Prefix 和 TotalVis 公理，其关注点在于维护全局的一致顺序。系统通过 HLC 的推进与等待机制来实现。对于前缀一致性（PC）、快照隔离（SI）以及可串行化（SER）级别的事务，路由服务器在事务开始时会通过调用 HLC 的  $Now()$  接口获取当前的全局 HLC 时间作为事务的快照时间  $snapshot\_time$ 。为了防止读取操作“跳过”那些逻辑上发生在前但尚未完成提交的事务，分片服务器侧引入了等待机制：当处理读请求时，如果分片发现目标键正在被另一个处于 Prepare 阶段的事务修改（即已分配提交时间戳但尚未最终 Commit），且该准备事务的  $prepare\_time \leq snapshot\_time$ ，分片服务器会暂时阻塞当前的读请求，直到该准备事务完成提交或中止。这一机制有效地对齐了物理上的提交顺序与逻辑上的可见性顺序。

对于 NoConflict 公理，即防止写冲突，系统利用两阶段提交（2PC）中的写验证（Certification）机制来保证。在事务的 Prepare 阶段，分片服务器会检查事务的写集（WSet）。如果检测到在当前事务的执行区间  $[sts, cts]$  内，存在另一个已经提交的事务修改了相同的键，则当前事务必须中止（Abort）。这一冲突检测逻辑确保了任何成功提交的事务都不仅观察到了与其冲突的、在抽象执行（Arbitration）序中排在前面的所有事务，而且没有被并发的冲突事务所覆盖，从而消除了丢失更新现象。

最后，对于可串行化（SER）级别，需要额外满足 TotalVis 公理，因此仅依赖 SI 式的写集认证是不够的。为了让成功提交的事务能够被解释为位于某个全序之中，同时又不遗漏其逻辑上的前驱事务，系统必须进一步约束读结果与全局

顺序之间的关系。围绕这一目标，理论上至少可以考虑以下三类方案。

第一类方案是基于集中式时间戳的验证协议<sup>[42]</sup>。该方案把可串行化判定集中到统一的时间戳序上：事务启动时获取快照时间，提交时检查其活跃区间内是否存在会破坏串行化解释的并发提交。若在  $[T.sts, T.cts]$  区间内有新的冲突事务提交，则当前事务中止。该方法的优点是语义直接、分析相对简洁；但在分布式场景下，它通常要求较强的集中协调能力，容易把时间戳分配和提交验证汇聚到单一控制点，从而带来额外的协调瓶颈。

第二类方案是基于锁的串行化协议<sup>[43]</sup>。该方案将快照读取与严格两阶段锁（S2PL）结合起来，通过共享锁和排他锁控制读写访问，并把锁持有到提交后统一释放。其优势在于串行化语义较为直观，许多冲突能够在执行阶段被直接阻止；但其代价也十分明显，即并发度下降、阻塞范围扩大，并且在跨分片事务场景中需要更复杂的分布式锁管理与死锁处理机制。因此，虽然该方案在理论上能够满足 TotalVis 与 NoConflict，但与本文追求在混合隔离级别下兼顾并发性与可扩展性的目标并不完全一致。

第三类方案是基于混合逻辑时钟与读检查集的分布式验证机制<sup>[44]</sup>。这一方案保留了 MVCC 与时间戳驱动的整体框架，同时把 SER 所需的额外约束集中到“读集合的持续追踪与提交前认证”上。相比前两类方案，它不要求所有事务都转化为悲观锁协议，也不需要把全部可串行化判断集中到单一点完成，更适合本文采用的分片式分布式架构。因此，本文最终采用这一方案作为 SER 事务的实现基础。具体而言，该机制包含以下三个关键环节：

- **混合逻辑时钟（HLC）与仲裁顺序：**系统使用 HLC 为事务分配单调递增的时间戳，从而为所有事务提供了全局单调的仲裁顺序（Arbitration 关系，即 AR 关系），这为全局可见性与因果定序奠定了坚实的基础。
- **快照读与检查集（Checkset）追踪：**SER 事务在开启时会获取当前的集群全局 HLC 时间作为其快照时间（ $sts$ ）。与弱隔离级别通常只跟踪写操作不同，SER 事务在执行过程中，会将其读取的所有键一并加入检查集中进行持续跟踪。
- **提交认证与冲突检测：**在两阶段提交的准备阶段（Prepare），分片服务器会执行严格的读集认证与冲突检测。服务器会检查是否存在已提交的并发事务  $T'$ ，使得其提交时间晚于当前事务的快照时间（即  $cts(T') > sts$ ），并且

该事务  $T'$  修改的数据与当前事务的检查集（同时包含读键和写键）产生了交集。如果检测到此类冲突，当前事务将被强制中止（Abort）。

这一检查集机制确保了 SER 事务不会在保留旧快照结果的同时忽略并发前驱的影响。在形式化语义模型中，SER 要求事务的可见性关系能够覆盖其应当观察到的全部 AR 前驱；而在系统实现中，这一要求被落实为“若存在晚于  $sts$  提交、且与当前事务检查集发生交集的并发事务，则当前事务不得成功提交”。因此，SER 相比 SI 的额外代价，正是对读集合的持续维护和更严格的提交前认证；而它所获得的收益，则是能够排除写偏斜等仅靠写集认证无法禁止的异常。

从比较的角度看，上述三类方案分别代表了“集中验证”“悲观锁控制”和“分布式读集认证”三种不同的实现思路。前两者在理论上都能支持 SER，但要么对集中控制依赖较强，要么会明显压缩系统并发度；而第三种方案能够更自然地与本文已经采用的 HLC、MVCC、两阶段提交和分片执行框架耦合起来。因此，把三种方案并列展开，不仅有助于说明 SER 语义本身的实现难点，也能够更清楚地解释本文为什么最终选择基于 HLC 与检查集的方案作为操作语义的落地路径。

基于上述形式语义分析，本系统在各隔离级别的实现过程中在不同的系统层面采用了对应的定制化机制。表 3-2 总结了 CC、PC、PSI、SI 和 SER 这五个主要隔离级别的相对于 RA 的核心定制点。

为了帮助读者更直观地理解这些定制点，可以把不同隔离级别下的事务执行差异概括为对同一条事务路径逐步增加约束。对 CC 而言，系统重点保证因果相关的先行事务不会被遗漏，因此关键在于客户端维护依赖时间戳并把它传播到后继事务；对 PC 而言，系统不追踪因果依赖，而是要求每次读取都落在某个全局一致前缀上，因此核心机制转化为基于 HLC 的快照选取和对未决前驱事务的等待；PSI 可以看作在 CC 的基础上增加写冲突认证，SI 则是在 PC 的基础上增加同类写冲突认证，因此二者分别把“因果一致快照”和“前缀一致快照”与 NoConflict 公理结合起来；SER 则进一步要求读集合在提交时仍与全局顺序相容，因此系统必须把读信息纳入认证范围，形成比 SI 更严格的提交条件。

如果从事务生命周期的角度重新观察这一差异，还可以得到一个更细的结论：不同隔离级别并不是简单对应于“强或弱”的静态标签，而是对应于系统在不同阶段愿意承担多大验证成本。CC 和 PSI 把更多工作放在事务启动前后的依

表 3-2 隔离级别实现的核心定制点

| 隔离级别       | 客户端定制                                                                     | 路由层定制                              | 分片层定制                                                                         |
|------------|---------------------------------------------------------------------------|------------------------------------|-------------------------------------------------------------------------------|
| <b>CC</b>  | 维护依赖时间戳<br>$dep \leftarrow \max(dep, cts)$ ; 事务开始时设置 $sts \leftarrow dep$ | 从客户端读取 $dep$ 作为事务快照时间; 分配 HLC 时间戳  | MVCC 多版本读取; 检查读-写依赖; 保证因果顺序                                                   |
| <b>PC</b>  | 普通时间戳维护; 不需要特殊依赖追踪                                                        | 调用 $Now()$ 获取全局 HLC 时间作为快照时间 $sts$ | 等待机制防止读操作跳过处于 <b>Prepare</b> 的事务; 对齐提交顺序                                      |
| <b>PSI</b> | 同 CC 的依赖时间戳追踪; 维护 $dep$                                                   | 同 CC 的机制; 传递 $dep$ 给分片层            | 因果一致性验证 + 写集冲突检测 (Certification); 防止丢失更新                                      |
| <b>SI</b>  | 普通时间戳; 不需要依赖追踪                                                            | 同 PC 机制; 获取全局 HLC 时间               | 一致性快照验证 + 写集冲突检测; NoConflict 检查; 防止丢失更新                                       |
| <b>SER</b> | 快照时间戳维护                                                                   | 分配 HLC 时间戳作为快照 $sts$ ; 为事务建立全局定序   | 检查集追踪 (读键 + 写键); 严格读集认证; 检查条件 $cts(T') > sts$ 且 $T'$ 与 <b>checkset</b> 有交集时中止 |

赖传播上, 强调“与我因果相关的事务不能缺失”; PC 和 SI 则把重点放在读取阶段的快照解释和等待机制上, 强调“我看到的历史必须是某个前缀”; SER 则进一步把压力推进到提交阶段, 通过检查集认证保证“我所依据的读结果在最终全序下仍然成立”。这种视角有助于理解混合隔离级别系统的一个核心特点: 系统并不是为所有事务统一执行最强的控制, 而是根据事务的语义要求, 把验证工作有选择地放到不同阶段、不同层面上执行。

从表 3-2 可以看出, 实现从弱隔离级别 (CC) 到强隔离级别 (SER) 的核心策略包括:

1. **依赖追踪范围扩展:** 从无依赖追踪 (PC、SI) 到因果依赖追踪 (CC、PSI) 再到完整检查集追踪 (SER)。依赖追踪的信息越丰富, 系统能够更精确地检测冲突。
2. **快照时间戳来源多元化:** 依赖型级别使用客户端维护的依赖时间戳, 前缀型级别使用全局 HLC 时间, 而可串行化级别需要这个时间戳建立全局的单调定序。
3. **冲突检测力度递进:** 从基本的写冲突检测 (SI、PSI) 升级到完整的读写冲突交集检测 (SER), 并通过等待机制或中止机制来强制满足理论公理。
4. **验证位置与时机精细化:** 不同级别在 **Prepare**、**Commit**、或 **Read** 阶段选择

不同的验证策略，形成了从乐观并发控制到悲观读验证的完整谱系。

### 3.5 本章小结

本章详细阐述了面向混合隔离级别的分布式数据库系统操作语义设计。首先给出了原型系统的体系结构建模，明确客户端、路由节点、配置节点和分片节点之间的职责划分；随后介绍混合逻辑时钟在全局定序中的作用；最后围绕不同隔离级别所对应的公理组合，说明读写缓冲、依赖追踪、等待机制和冲突检测等核心语义约束如何在系统中落地。

本章的讨论为后续的  $\text{TLA}^+$  验证与系统原型实现提供了操作语义层面的基础。



## 第四章 支持混合隔离级别机制的 $TLA^+$ 建模与验证

### 4.1 事务操作语义的 $TLA^+$ 表达

第二章已经介绍了  $TLA^+$  与 TLC 的基本概念。本章在此基础上进一步面向本文提出的事务模型，给出事务操作语义的  $TLA^+$  表达方式。形式化规约仍采用“初始状态 *Init* + 状态转移 *Next* + 安全断言”的组织范式，但其中涉及的状态变量、动作定义与约束条件均围绕混合隔离级别事务的执行过程展开。

在状态建模上，本文将事务执行过程抽象为一个离散状态机。系统状态不仅包含全局键值存储及其版本历史，还包含事务缓冲区、事务快照、操作历史以及客户端依赖信息等辅助变量。通过这种建模方式，系统在任意执行时刻的可见状态都能够被形式化描述，从而为后续公理的检查提供统一的语义基础<sup>1</sup>。

### 4.2 公理和隔离级别约束

在前文中，本文已经介绍了  $TLA^+$  规约语言及 TLC 模型检查器的基本概念与设计范式。

在本节中，本文进一步说明如何利用  $TLA^+$  构建一个面向混合隔离级别操作语义的严格验证框架。在这一框架下，形式化约束的构造主要包括两个方面：其一是将事务执行历史转化为抽象执行中的 VIS 与 AR 关系；其二是将一致性公理与隔离级别约束转化为可由 TLC 检查的逻辑断言。

#### 4.2.1 状态变量与动作定义

##### 状态模型

定义操作语义的第一步是构造状态模型。本文使用一组状态变量来表示系统的当前状态，这些变量共同描述事务状态、数据项取值、版本可见性以及相关

---

1 项目模块代码提供在 <https://github.com/ZhangJihua0327/mil-tla-verify> 仓库中。

依赖关系。通过对这些组件作出明确约束，可以较为准确地刻画系统在任意时刻的运行状态。

重要的变量有：

- `store`: 全局键值存储，模拟数据库的持久化状态。它是一个映射，键是 `Keys`，值是版本历史（包含值、提交时间戳 `cts`、写入者）。
- `tx_buffer`: 事务的本地写缓冲，模拟事务在提交前对数据的本地修改。
- `tx_snapshot`: 事务的读取快照版本，决定了事务能看到哪些数据。
- `op_seq`: 记录操作的历史序列，用于后续的公理化验证。

通过上述变量，可以构建一个较为完整的状态模型，并据此进一步形式化描述事务操作语义。

## 核心状态转换动作 (Transition Actions)

操作语义的核心逻辑可归纳为三个主要的 TLA<sup>+</sup> 动作定义，分别对应事务开始、操作执行与事务提交三个阶段。三类动作并不是彼此孤立的局部规则，而是共同构成了事务从建立可见快照、逐步消费操作序列，到最终完成校验与提交的完整状态演化过程。按照这一顺序展开说明，不仅能够更清楚地呈现模型中的状态转移链条，也有助于解释这些动作如何共同支撑后续的一致性公理验证。

### A. 事务开始 ( $StartTx(t)$ )

$StartTx(t)$  形式化描述了事务获取读取视图 (Read View) 的过程。该动作首先根据事务的隔离级别  $Workload[t].level$  为  $tx\_snapshot[t]$  赋值，再将事务状态  $tx\_status$  更新为 “Active”，从而使事务进入可执行状态。不同隔离级别对应不同的快照来源：SI (Snapshot Isolation) 直接读取当前全局提交时钟  $shard\_ct$ ，RA (Read Atomicity) 则继承客户端最近一次提交时间  $client\_last\_commit$ 。更一般地说，PC、SI 和 SER 从全局提交时钟获取一致快照，CC 与 PSI 继承会话依赖，而 RA 仅依赖客户端最近一次提交时间。由此可见，事务在开始阶段绑定的快照来源不同，后续读操作可访问的版本边界也随之不同。换言之，不同隔离级别在可见性上的差异，在模型中首先体现为事务启动阶段对快照来源的不同选择。

## B. 执行操作 ( $ExecuteOp(t)$ )

$ExecuteOp(t)$  给出了读写操作的具体语义，并依据  $Workload$  中预定义的操作类型进行分支。对于读操作，模型首先检查本地写缓冲  $tx\_buffer[t][op.key]$ ，从而保证事务内部满足 Read-Your-Own-Writes 语义；若本地不存在对应写入，则调用辅助算子  $MaxVersion(store[op.key], tx\_snapshot[t])$ ，从全局存储的版本历史中选取提交时间戳不超过事务快照的最新版本，并将读取结果追加到  $op\_seq$  以供后续验证。对于写操作，模型并不立即修改全局  $store$ ，而是先将结果写入  $tx\_buffer$ ，同时将该写操作记录到  $op\_seq$  中。这里需要强调的是，读操作和写操作在模型中承担的并不是对称职责：前者负责在“局部写集”和“快照可见版本”之间解析事务当前可读到的值，后者则负责积累尚未提交的局部更新，为提交阶段的原子写入预先组织好输入。这样的定义保留了两个最关键的语义点：一是事务内部的本地优先读取，二是基于快照上界的版本选择。由此，版本可见性被归约为一个可由 TLC 直接枚举和检查的选择过程，同时也使本地缓冲与全局版本历史之间的分工更加清晰。

## C. 提交事务 ( $CommitTx(t)$ )

$CommitTx(t)$  则形式化描述了提交协议，其中同时包含并发控制与原子性要求。模型首先通过  $HasConflict$  执行冲突检测，并借助谓词  $IsConcurrent(t\_other)$  识别与当前事务并发的已提交事务；若存在其他事务的写入集与当前事务的检查集（Checkset/ReadSet）发生交叉，则说明乐观并发控制（OCC）的验证失败，此时事务状态被更新为“Aborted”。若冲突检测通过，模型便进入提交路径：一方面递增全局时间戳  $shard\_ct$ ，另一方面将  $tx\_buffer$  中的数据以原子方式合并到全局  $store$ ，生成带有新时间戳的版本，并同步更新客户端依赖  $client\_dep$ 。这一设计的关键之处在于，它将“是否允许事务提交”和“提交后如何改变全局状态”放在同一个动作中统一处理，从而避免把语义上应当同时成立的步骤拆散为多个松散的状态变化。在这一阶段，模型通过  $HasConflict$  谓词统一表达 PSI、SI 与 SER 等隔离级别所需的并发校验，并以单个状态跃迁完成提交时间分配与新版本写入。由于  $TLA^+$  以状态转移为核心，上述定义能够将“冲突检测”与“原子提交”纳入同一条可验证路径，从而避免在正文中展开与证明主线

关系不大的工程性细节，也使提交阶段的形式化表达更接近实际系统中的整体提交语义。

#### 4.2.2 抽象执行关系与公理约束

抽象执行  $\mathcal{A} = (\mathcal{H}, \text{VIS}, \text{AR})$  中， $\mathcal{H}$  表示操作历史。由于状态模型部分已经涉及  $\mathcal{H}$  的基本内容，本节不再单独展开，而是重点构建 VIS 与 AR 的形式化表达。图 4-1 展示了这两个关系的形式化表示方式。

$$\begin{array}{c}
 \text{MODULE } \text{VisAndAr} \\
 \hline
 \text{Derived\_AR} \triangleq \\
 \{ \langle t1, t2 \rangle \in \text{CommittedTrns} \times \text{CommittedTrns} : \\
 \quad \text{tx\_commit\_time}[t1] < \text{tx\_commit\_time}[t2] \\
 \} \\
 \\
 \text{Derived\_VIS} \triangleq \\
 \{ \langle t1, t2 \rangle \in \text{CommittedTrns} \times \text{CommittedTrns} : \\
 \quad \vee \wedge t2 \neq \text{InitTx} \\
 \quad \quad \wedge \text{Workload}[t2].\text{level} \in \{ \text{"RA"}, \text{"CC"}, \text{"PC"}, \text{"PSI"}, \text{"SI"} \} \\
 \quad \quad \wedge \text{tx\_commit\_time}[t1] \leq \text{tx\_snapshot}[t2] \\
 \quad \vee \wedge (t2 = \text{InitTx} \vee \text{Workload}[t2].\text{level} = \text{"SER"}) \\
 \quad \quad \wedge \langle t1, t2 \rangle \in \text{Derived\_AR} \\
 \} \\
 \hline
 \end{array}$$

图 4-1 关系 VIS 和 AR 的形式化表达

这里的核心思想是将事务历史中的可见性关系与仲裁关系明确写成可检验的形式化对象，使前文给出的一致性公理不再停留于抽象的数学关系层面，而能够与模型检查中的状态约束形成一一对应。

在定义抽象执行  $\mathcal{A}$  之后，还需要进一步对公式 2-4 中的公理进行形式化表达，才能将前文给出的数学定义真正落实为 TLC 可检查的逻辑约束。换言之，若说 VIS 与 AR 给出了事务历史的结构，那么这些公理所描述的就是该结构必须满足的合法性条件。图 4-2 展示了这些公理的形式化表达；在此基础上，便可进一步定义不同隔离级别对应的约束，并把“抽象一致性要求”转化为“模型检查中的可判定性质”。

$$\begin{array}{l}
\text{MODULE } Axioms \\
\hline
IntAxiom(t) \triangleq \\
\quad \forall i \in OpsOf(t) : \\
\quad \quad op\_seq[t][i].type = "R" \Rightarrow \\
\quad \quad \text{LET } prev\_ops \triangleq \\
\quad \quad \quad \{j \in 1 \dots (i-1) : op\_seq[t][j].key = op\_seq[t][i].key\} \\
\quad \quad \text{IN } prev\_ops \neq \{\} \Rightarrow \\
\quad \quad \quad \text{LET } max\_j \triangleq \text{CHOOSE } j \in prev\_ops : \forall k \in prev\_ops : j \geq k \\
\quad \quad \quad \text{IN } op\_seq[t][i].val = op\_seq[t][max\_j].val \\
\\
ExtAxiom(t, VIS, AR) \triangleq \\
\quad \forall x \in Keys, \\
\quad \quad v \in Values : \\
\quad \quad T\_reads\_ext(t, x, v) \Rightarrow \\
\quad \quad \text{LET } vis\_writes \triangleq \\
\quad \quad \quad \{tp \in CommittedTxns : \\
\quad \quad \quad \quad \langle tp, t \rangle \in VIS \wedge LastWriteIndex(tp, x) > 0 \\
\quad \quad \quad \quad \} \\
\quad \quad \text{IN } T\_writes(MaxAR(vis\_writes, AR), x, v) \\
\\
TransVisAxiom(t, VIS) \triangleq \\
\quad \forall tp, s \in CommittedTxns : \\
\quad \quad (\langle tp, s \rangle \in VIS \wedge \langle s, t \rangle \in VIS) \Rightarrow \langle tp, t \rangle \in VIS \\
PrefixAxiom(t, VIS, AR) \triangleq \\
\quad \forall tp, s \in CommittedTxns : \\
\quad \quad (\langle tp, s \rangle \in AR \wedge \langle s, t \rangle \in VIS) \Rightarrow \langle tp, t \rangle \in VIS \\
NoConflictAxiom(t, VIS, AR) \triangleq \\
\quad \forall tp \in CommittedTxns : \\
\quad \quad (WriteConflict(tp, t) \wedge \langle tp, t \rangle \in AR) \Rightarrow \langle tp, t \rangle \in VIS \\
TotalVisAxiom(t, VIS, AR) \triangleq \\
\quad \forall tp \in CommittedTxns : \langle tp, t \rangle \in AR \Rightarrow \langle tp, t \rangle \in VIS \\
\hline
\end{array}$$

图 4-2 公理的形式化表达

图 4-2 中各项公理并不是彼此孤立的局部条件，而是与前述的 VIS 和 AR 关系共同构成完整的抽象执行约束。也正因为如此，模型验证并不是单纯检查某个局部谓词是否成立，而是检查整个执行历史是否同时满足可见性关系、仲裁顺序以及一致性公理之间的联合约束。

不同于面向具体业务逻辑的程序语言， $TLA^+$  更关注状态转移结果本身。运行  $TLA^+$  代码时，TLC 模型检查器会穷举所有可能的状态转移路径，并检查这些路径是否满足预先定义的安全属性。通过配置不同的安全属性，便可以验证系统在不同隔离级别下是否满足相应约束，从而验证模型设计的正确性。

### 4.3 模型检查实验与结果

测试用例只需通过 `INSTANCE MixedIsolation WITH ...` 语法注入负载，验证引擎即可启动运行。在验证过程中，TLC 模型检查器会探索所有通过 `Next` 产生的合法状态转移路径。TLC 的穷举能力使网络延迟、乱序提交与快照

交错等情形均能够被覆盖。一旦在某个微观并发场景下发现生成的执行图违反 `OperationalSafety == IsConsistentExecution` 断言，模型检查器就会立即停止，并输出精确的反例路径追踪报告（Error Trace File），以协助开发者定位并修复并发漏洞。

本文在实验阶段采用 **SmallBank** 负载对混合隔离级别模型进行实例化，原始测试用例见附录 A.1。通过注入该负载，TLC 模型检查器可以检验系统在该场景下是否满足所定义的安全属性，并据此发现潜在的并发问题。模型检查结果显示，在当前 **SmallBank** 负载与参数设置下，系统满足预先定义的安全属性，且未发现违反一致性要求的执行路径。这表明该操作语义在该负载下与目标隔离级别约束是一致的。

该测试用例给出了一个最小但具有代表性的混合隔离级别验证场景。图中定义了两个并发事务，其中事务  $T_1$  以 PSI 隔离级别读取 `checking` 与 `savings` 两个账户并更新储蓄账户余额，事务  $T_2$  则以 SER 隔离级别对相同数据项执行读取与写入操作。这样一组负载既保留了 **SmallBank** 场景中典型的账户读写依赖，又能够显式触发跨事务可见性、检查集冲突以及不同隔离级别交织执行下的一致性约束，因此适合作为 TLC 模型检查的实例化输入。

该测试用例给出了一个最小但具有代表性的混合隔离级别验证场景。图中定义了两个并发事务，其中事务  $T_1$  以 PSI 隔离级别读取 `checking` 与 `savings` 两个账户并更新储蓄账户余额，事务  $T_2$  则以 SER 隔离级别对相同数据项执行读取与写入操作。这样一组负载既保留了 **SmallBank** 场景中典型的账户读写依赖，又能够显式触发跨事务可见性、检查集冲突以及不同隔离级别交织执行下的一致性约束，因此适合作为 TLC 模型检查的实例化输入。

图 4-3 展示了 TLC 模型检查器的终端输出示例，其中包含模型检查进度、状态生成数量、已发现状态数量以及最终检查结果。由该结果可知，TLC 模型检查器成功完成了模型检查，且未发现任何违反安全属性的状态。这进一步表明，在 **SmallBank** 负载下，系统的操作语义能够保证事务满足目标隔离级别的要求。

按照同样的方式，还可以在更多测试集合上重复上述过程，以检验当前操作语义在不同负载下是否持续满足所定义的安全属性，并进一步排查潜在的并发问题。若在这些负载条件下均未发现违反约束的执行路径，则可以说明该操作语义与相应隔离级别要求保持一致。

```
java.exe -cp
↪ c:\Users\zhang\.vscode\extensions\tlaplus.vscode-ide-2026.3.
↪ -XX:+UseParallelGC -DTLA-Library= tlc2.TLC
↪ Benchmark_SmallBank.tla -tool -modelcheck -config
↪ c:\Users\zhang\Documents\Project\mil-tla-verify\Benchmark_Sm

Progress(11) at ____: 68 states generated (6,845 s/min), 42
↪ distinct states found (4,228 ds/min), 0 states left on
↪ queue.
68 states generated, 42 distinct states found, 0 states left on
↪ queue.
The depth of the complete state graph search is 11.
The average outdegree of the complete state graph is 1 (minimum
↪ is 0, the maximum 2 and the 95th percentile is 2).
Finished in 599ms at ____
```

图 4-3 TLC 模型检查器的终端输出示例

## 4.4 本章小结

本章围绕支持混合隔离级别机制的  $TLA^+$  建模与验证展开。首先给出了事务操作语义的  $TLA^+$  表达方式，说明状态变量与核心动作如何共同描述事务执行过程；其次构建了抽象执行关系、一致性公理与隔离级别约束的形式化断言；最后结合模型检查实验及结果分析，验证了本文所设计的操作语义能够保证事务满足目标隔离级别的要求，并为后续原型系统实现提供形式化依据。



## 第五章 分布式键值数据库原型的架构设计与实现

本章将基于前序章节中关于分布式数据库的建模和分析，以及经  $TLA^+$  验证过的支持混合隔离级别的分布式数据库操作语义，设计并实现一个数据库原型系统，用于验证该操作语义相较于只支持单一隔离级别的操作语义在性能上的优势。

### 5.1 原型系统概述

前文提出的事务操作语义与并发控制机制，最终需要落到可运行的系统中才能检验其可行性。基于这一点，本文实现了一个分布式键值数据库原型。该原型由路由节点、配置节点和分片节点组成，支持多版本并发控制、跨分片事务协调以及多种隔离级别的混合执行。它并不以产品化为目标，而是作为研究载体，用于验证核心机制能否落地实现，并为后续实验提供可重复的运行环境。

具体来说，该原型主要承担三项工作：验证第三章提出的系统操作语义能否转化为可执行机制，检验第四章中的形式化约束能否与关键模块设计保持一致，以及通过实验比较不同隔离级别配置下的吞吐量与延迟表现。基于这一定位，本章先说明系统需求与实现边界，再展开架构设计和关键模块实现。

### 5.2 需求分析

该原型的设计目标比较明确：在保证目标事务语义的前提下，观察混合隔离级别执行相较于统一隔离级别方案是否具有性能优势。围绕这一目标，下面从功能需求和非功能需求两个方面说明系统需要具备的能力。

#### 5.2.1 功能需求

功能需求定义了系统必须完成的具体任务和行为。基于验证混合隔离级别性能优势的目标，系统的主要功能需求如下：

1. **基本数据操作**：本原型系统作为一个键值（Key-Value）数据库，需支持基本的读（Get）、写（Put）、增（Insert）、删（Delete）四种数据操作。数据模型以键值对（Key-Value）形式存储，不支持复杂的嵌套文档结构。
2. **分布式集群管理**：
  - **分片（Sharding）**：支持将数据水平切分并分布在多个分片节点上。
  - **路由（Routing）**：客户端通过路由节点访问数据，路由节点根据分片键将请求转发至正确的分片。
3. **事务管理**：
  - **事务生命周期**：支持事务的开启（Start）、提交（Commit）和中止（Abort）。
  - **跨分片事务**：支持涉及多个分片的分布式事务，保证原子性。
4. **混合隔离级别支持**：这是本系统的核心需求。系统允许客户端在开启事务时指定该事务的隔离级别。系统应至少支持以下几种隔离级别的混合运行：
  - **前缀一致性（Prefix Consistency, PC）**
  - **并行快照隔离（Parallel Snapshot Isolation, PSI）**
  - **快照隔离（Snapshot Isolation, SI）**
  - **可串行化（Serializable, SER）**

系统需要确保在上述隔离级别混合运行时，事务能够在要求的隔离级别下成立，并保证系统行为与形式化语义保持一致。

## 5.2.2 非功能需求

非功能需求涉及系统的性能、可靠性等方面：

1. **高性能**：由于采用了全内存存储，系统应具有较低的读写延迟和较高的吞吐量。
2. **可扩展性**：系统应支持水平扩展，通过增加分片节点来线性提升存储和处理能力。
3. **可观测性**：为了验证性能优势，系统必须提供详细的性能监控指标（如延迟、吞吐量、冲突率等），以便进行后续的实验对比。

### 5.2.3 用例分析

系统主要涉及两类角色：客户端用户（Client）和系统管理员（Admin）。

- **客户端用户：**主要进行数据的读写操作和事务处理。他们可以发起普通读写请求，也可以开启特定隔离级别的事务进行一系列操作。
- **系统管理员：**负责集群的配置，如添加分片、配置路由规则等。

用例图 5-1 展示了系统的核心用例。其中，“执行混合隔离级别事务”是本系统的特色用例，用户在开启事务时可以选择所需的隔离级别（PC, PSI, SI, SER）。

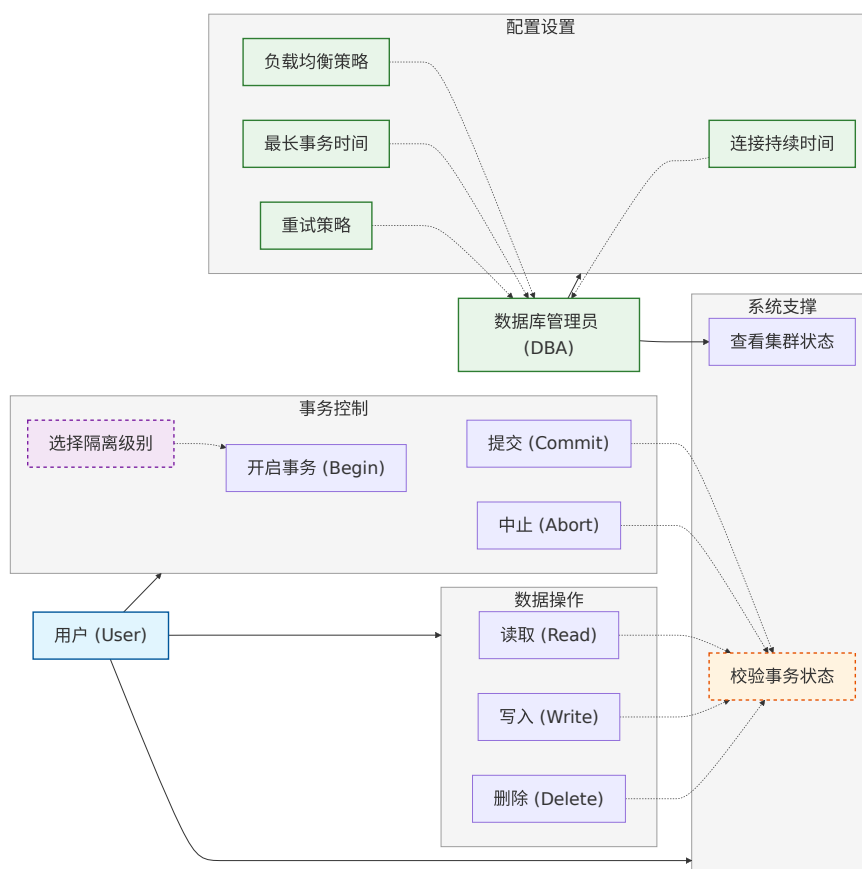


图 5-1 系统用例图

## 5.3 系统设计

基于上述需求分析，本节将对数据库原型系统进行总体架构设计和模块设计。

### 5.3.1 总体架构设计

本系统采用经典的 Shared-nothing 分布式架构<sup>[45]</sup>，参考 MongoDB 的集群模型，主要由三类组件构成：路由节点（Router）、配置节点（Config Server）和分片节点（Shard Server）。整体组件关系如图 5-2 所示。

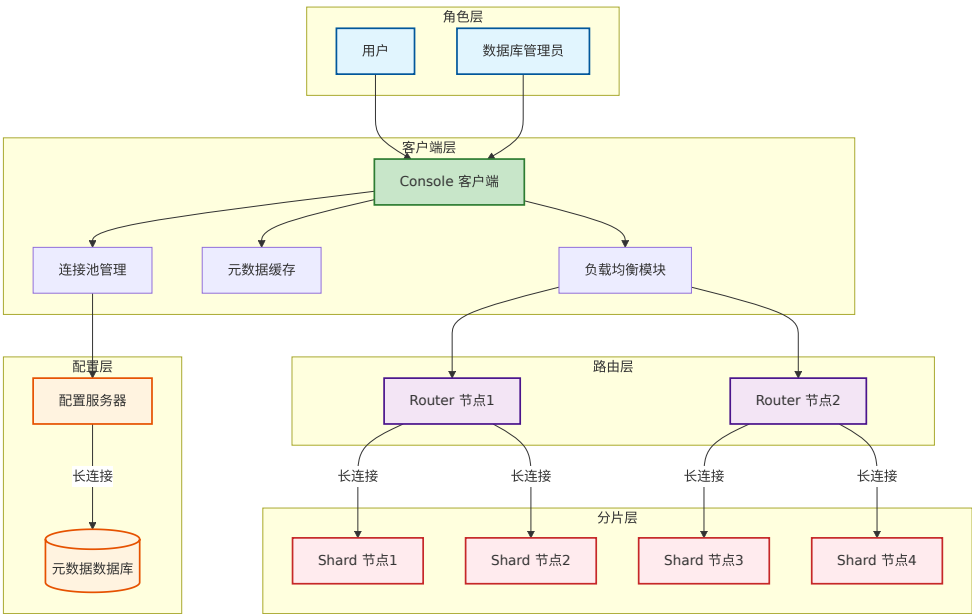


图 5-2 系统总体架构图

各组件职责如下：

- **路由节点（Router）：**作为集群的统一入口，对外提供以 MySQL 或自定义协议的接口。路由节点不存储实际数据，只缓存元数据（Metadata）。它负责解析客户端请求，根据分片键（Shard Key）和路由表（Routing Table）将请求路由到具体的分片节点。对于跨分片事务，路由节点充当事务协调者（Coordinator）的角色，负责两阶段提交（2PC）的协调工作。
- **配置节点（Config Server）：**存储集群的元数据，包括分片信息、路由表规则、节点状态等。保证元数据的高可用和强一致性是配置节点的关键，但在本原型中，为了简化实现，可采用单点或简单的副本机制。
- **分片节点（Shard Server）：**负责实际数据的存储和操作执行。每个分片节点维护一部分数据，并包含本地的事务执行引擎和存储引擎。分片节点作为分布式事务的参与者（Participant），响应协调者的指令，并负责本地数据的并发控制（Concurrency Control）。

### 5.3.2 核心模块设计

#### 存储引擎设计

为了简化实现并聚焦于并发控制机制的验证，本原型采用全内存存储引擎。数据在内存中以键值对（**Key-Value**）的形式组织。为了支持多版本并发控制（**MVCC**），每个键对应的值不仅仅是单一数据，而是一个版本链（**Version Chain**）。

- **多版本数据结构**：每个数据项包含多个版本，每个版本携带创建该版本的事务时间戳（**Commit Timestamp**）和实际数据载荷。
- **内存索引**：使用哈希表（**Hash Map**）或跳表（**Skip List**）作为索引结构，实现  $O(1)$  或  $O(\log N)$  的快速查找。

#### 事务管理模块设计

事务管理模块是系统的核心，分为协调者模块（位于 **Router**）和参与者模块（位于 **Shard**）。

1. **混合逻辑时钟（HLC）**：系统采用 HLC 作为时间源。HLC 提供了单调递增的时间戳，用于定序事务和版本控制。
2. **并发控制协议**：为了支持混合隔离级别，系统采用基于时间戳的 MVCC 协议。
  - 对于 **Prefix Consistency (PC)** 事务，读取全局仲裁序列某个一致前缀上的可见版本，避免观察到带“空洞”的历史。
  - 对于 **Parallel Snapshot Isolation (PSI)** 事务，允许在不同的分片上读取到不同快照的数据，但在同一分片上满足 SI。
  - 对于 **Snapshot Isolation (SI)** 事务，在事务开始时获取快照时间戳（**StartTS**），读取  $\text{CommitTS} \leq \text{StartTS}$  的最新版本。
  - 对于 **Serializable (SER)** 事务，除了遵循 SI 的读规则外，还需进行冲突检测（如写偏斜检测），确保可串行化。
3. **分布式事务原子性**：采用两阶段提交（**2PC**）协议。
  - **Prepare 阶段**：协调者向所有参与者发送 Prepare 请求，参与者执行本地事务逻辑，锁定资源，并持久化（在内存中模拟）事务意向。

- **Commit/Abort 阶段:** 如果所有参与者 Prepare 成功, 协调者发送 Commit, 否则发送 Abort。

## 通信模块设计

节点间通信采用异步 I/O 模型 (如 RPC 或 TCP 长连接), 以减少网络通信开销。消息定义采用 Protobuf 等序列化格式, 确保高效传输。

## 5.4 系统实现

### 5.4.1 存储分片与分布式事务

在原型系统中, 分片 (Shard) 既负责数据的物理存储, 也承担事务执行过程中的版本管理与隔离级别约束。系统通过 shard 包下的 Shard 结构体封装分片状态机。本小节主要说明四部分实现: 分片内事务信息的组织方式、基于 MVCC 的读写流程、两阶段提交中的可见性漏洞修复, 以及单分片事务的快速提交优化<sup>1</sup>。

### 事务操作信息的分层与局部化管理

本系统采用了“全局协调计算, 局部状态下推”的事务信息管理范式。在事务生命周期初始化阶段 (TxStart), 事务的全局元数据 (如事务 ID、隔离级别和全局快照时间戳 snapshotTime) 由协调者 (Router) 统一获取并下发。这种设计能够让不同分片共享同一个全局时间基准, 进而支持前缀一致性 (PC)、快照隔离 (SI) 以及严格可串行化 (SER)。图 5-3 展示了全局事务信息的分发与下推流程, 协调者在事务开始时通过 RPC 将全局元数据广播至所有相关分片, 确保每个参与者节点都能基于同一时间线进行版本控制和并发调度。

相对地, 涉及具体分片的数据操作信息由分片内的 TxBufferManager 进行局部维护。考虑到未决事务的读写集可能带来较高的内存开销, 代码在此采用了懒加载 (Lazy Load) 机制: 只有当该分片真正执行首次 TxRead 或 TxWrite 时, 才会实例化对应事务的局部缓存 (TxBuffer)。这种分阶段存储方式减少了无效状

---

1 本文的代码提供在仓库 <https://github.com/ZhangJihua0327/let-mil-go> 中。

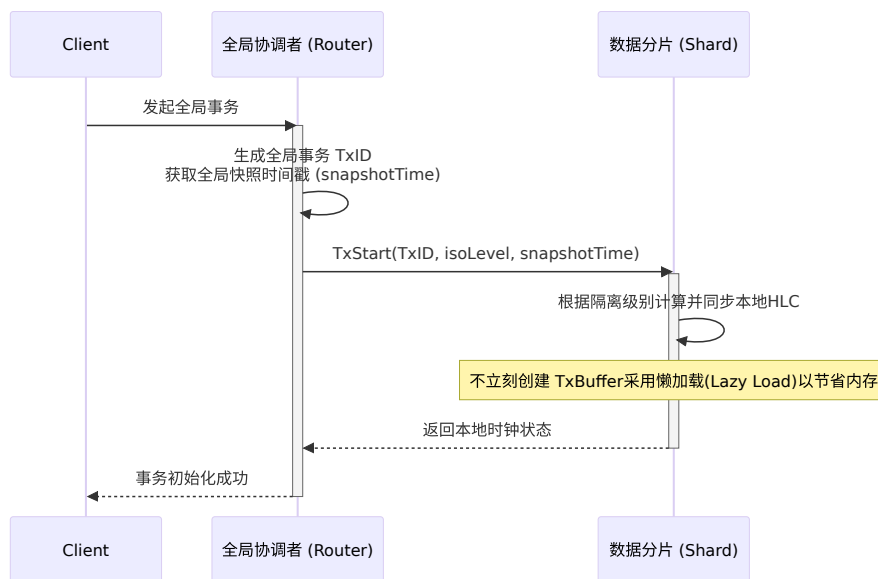


图 5-3 组件交互时序图

态分配，也使事务状态更多保留在实际参与执行的分片上。

## 基于 MVCC 引擎的多版本读写流转

分片底层的无锁化读并发依赖于多版本并发控制（MVCC）模型。以 TxRead 为例，代码在读取数据时，严格遵循“读己之所写（Read-Your-Own-Writes）”的内部一致性：系统首先探测本地事务缓冲区的 Write 集合，命中后判断记录是否带有删除标记（Deleted）。如果本地无修改记录，读请求将穿透至底层的 MVCCStore 操作对象，基于给定的 snapshotTime 定位并获取小于但最接近该时钟的可用历史版本。

此外，程序还为最高隔离级别（SER）引入了悲观的锁机制转换。如果事务的隔离级别为 SER，分片内部将直接通过 lockMgr.AcquireRead 强制索取分布式共享锁，在读写阶段即时屏蔽幻读和写倾斜缺陷，从而保证全局事务的严格串行化调度。

## 两阶段提交可见性漏洞 (Hole) 的消除

在标准的分布式两阶段提交（2PC）协议中，由于多个参与者节点的 Prepare 阶段无法做到绝对原子同频，常存在所谓的可见性漏洞（Visibility Hole）——即事务虽然已经预定将要提交，但若并发的读取操作携带了更大（或相同）的时间

戳进入，可能会因为读取到旧版本而破坏线性一致性。

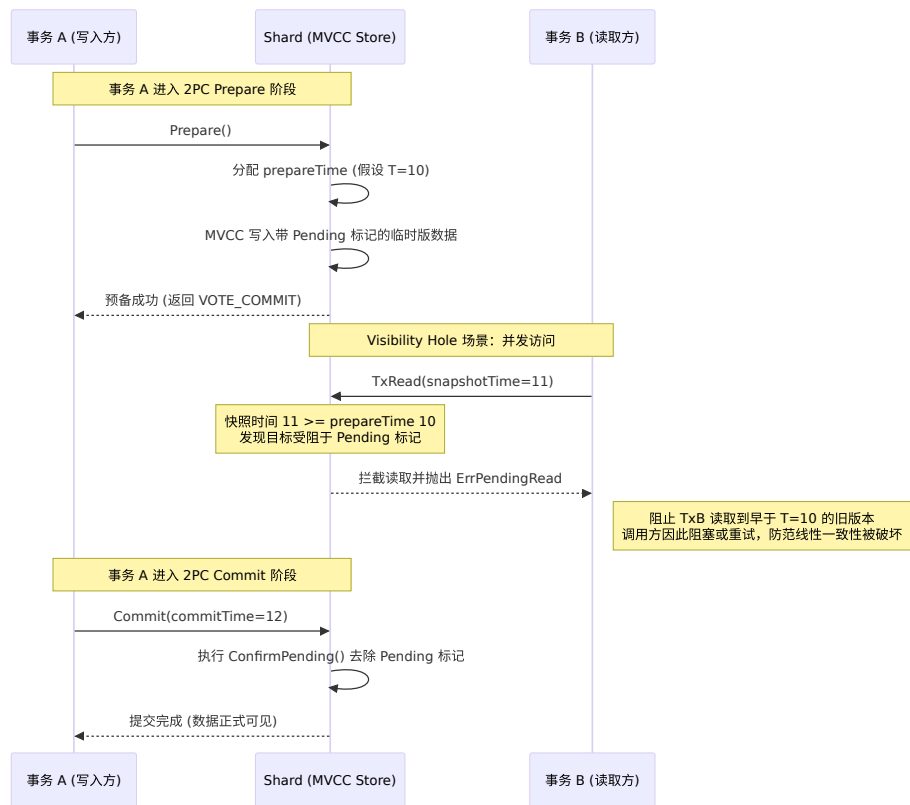


图 5-4 两阶段提交可见性漏洞示意图

图 5-4 展示了本组件处理两阶段提交可见性漏洞的方式。系统借助未决状态（Pending Status）与时间戳推进来约束该阶段的可见性：在 Prepare 阶段，系统根据混合逻辑时钟（HLC）派生本地的预备时间 prepareTime。此时系统并不直接覆盖旧版本，而是调用 store.Put 写入带有 Pending 标记的临时版本。该预写入过程为并发读操作提供了一个额外的判定条件：若并发读取请求的时间戳满足 snapshotTime ≥ prepareTime，且遇到 Pending 标记，底层将返回 ErrPendingRead，调用方随后进入等待或重试。待协调者最终作出提交决定后，Commit 通过 store.ConfirmPending(txId, commitTime) 将临时版本确认为正式版本。通过这一过程，预提交阶段与最终提交阶段之间的可见性得以衔接。

### 单分片快速提交 (QuickCommit) 优化

当系统规划器判定当前事务的作用域仅局限于某单一分片时，该组件会触发 QuickCommit 的快速提交路径，以规避 2PC 的网络协商与多次 RPC 的延迟开销<sup>[46-47]</sup>，如图 5-5。

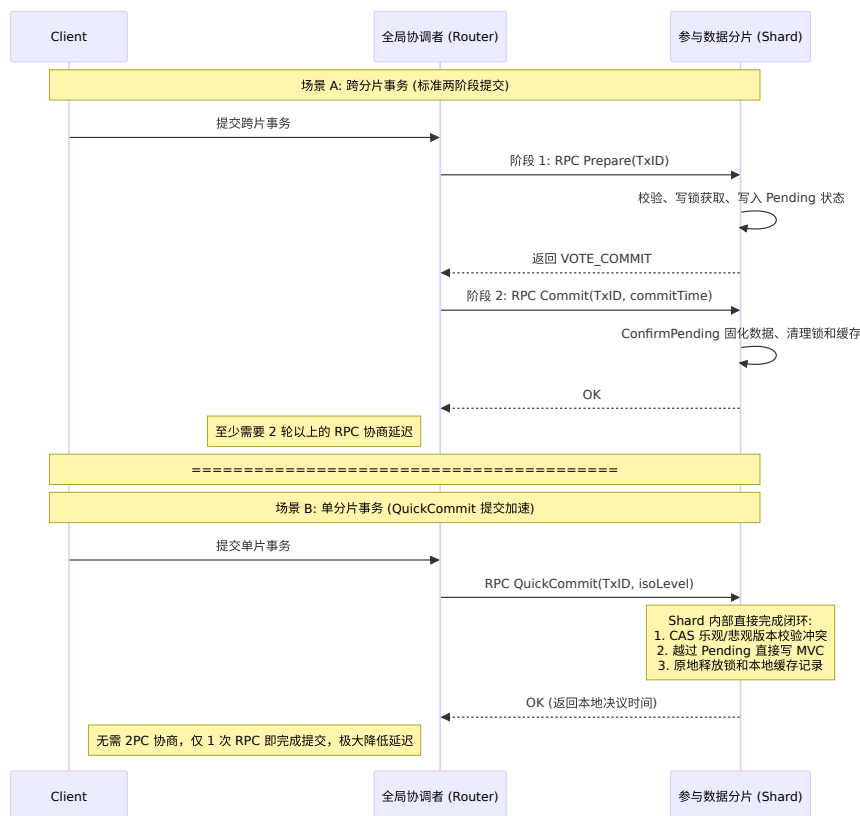


图 5-5 组件交互时序图

在 QuickCommit 的实现逻辑中，由于省略了 Prepare 阶段，事务提交流程被压缩为一次单分片内的原子变更。引擎首先通过 `clock.Tick()` 获取提交时间；随后，CAS 校验模块针对 PSI、SI 以及 SER 隔离级别执行乐观并发验证，以确认缓冲区中记录的 `OriginalVersion` 仍为当前最新版本，避免覆盖写入。在通过版本校验并获取必要的本地写锁后，QuickCommit 不再经过 Pending 预写入流程，而是将缓冲区中的数据直接以带有完整 `commitTime` 的正式版本写入 MVCC 存储层。这样一来，校验、写入、提交与释放锁都在单一片内完成，从而减少了单分片事务的提交路径。

### 5.4.2 混合逻辑时钟

在并发定序和版本可见性判断中，混合逻辑时钟（HLC）提供了统一的时间基准。本原型的 HLC 组件参考了开源项目 `glycerine/hlc` 的实现思路，并用 Go 语言实现了一个并发安全的时钟模块。

在具体的运行层面，该时钟模块重点实现了两个用于时钟状态机推进的基

石接口：

- **本地时间获取 (Now/Tick)**: 当节点（路由或分片）内部产生本地事件，如分配事务快照时间或构建局部预备时间时，系统会调用该接口。算法读取当前主机的物理时间，并与本地维护的历史最高 HLC 时间进行比较。如果物理时钟前进，则推进 WallTime 并将 Logical 清零；如果检测到时钟停滞或物理时钟回退，则保持当前物理时间上界不变，并通过原子操作递增逻辑计数器，从而生成单调递增的时间戳。
- **远程时钟同步 (Update/Receive)**: 在跨节点请求中，底层 RPC 拦截器会在报文头部 (Header) 附带发送方当前的 HLC 时间戳。接收端在解析应用层载荷之前，先调用时钟同步逻辑，对“本地维护的当前 HLC”“报文携带的远程 HLC”以及“本机当前物理时间”三者进行比较，并据此更新本地时间状态。该过程用于在节点交互过程中维持时间顺序的一致性。

基于上述时钟组件及其在 RPC 通信过程中的配合，系统能够在分布式节点之间维持一致的时间比较基础。相比集中式发号方式，这种去中心化时钟机制避免了额外的中心协调开销，也为基于 MVCC 的快照读取以及 SER 级别下的冲突检测提供了统一的时间边界。

### 5.4.3 通信原理与同步机制

在分布式数据库系统中，协调者 (Router) 与各个分片 (Shard) 之间存在高频内部通信。考虑到事务执行路径对网络开销较为敏感，系统没有采用传统的 HTTP RESTful 方式，而是使用基于 Protobuf (如 shardpb 包所示) 的 RPC 框架作为跨节点通信基础。

采用基于 Protobuf 的 RPC 协议，主要是出于通信效率方面的考虑。一方面，Protobuf 使用紧凑的二进制编码格式，相比 JSON 等文本协议可减少请求负载；另一方面，底层 RPC 框架依托 HTTP/2 的多路复用 (Multiplexing) 机制，使多个并发事务请求能够共享同一条底层 TCP 连接，从而降低连接建立与传输过程中的额外开销。

在事务处理的数据面之外，集群控制面中全局路由表、分片拓扑状态以及隔离级别元数据的同步同样至关重要。为了使分片节点能够以极低延迟感知集

群状态变更，系统在配置同步链路中抛弃了传统的客户端定时轮询（Polling）模型，转而引入了基于流式 RPC（Streaming RPC）的长连接实时主动推送机制，图 5-6 展示了其时序图。

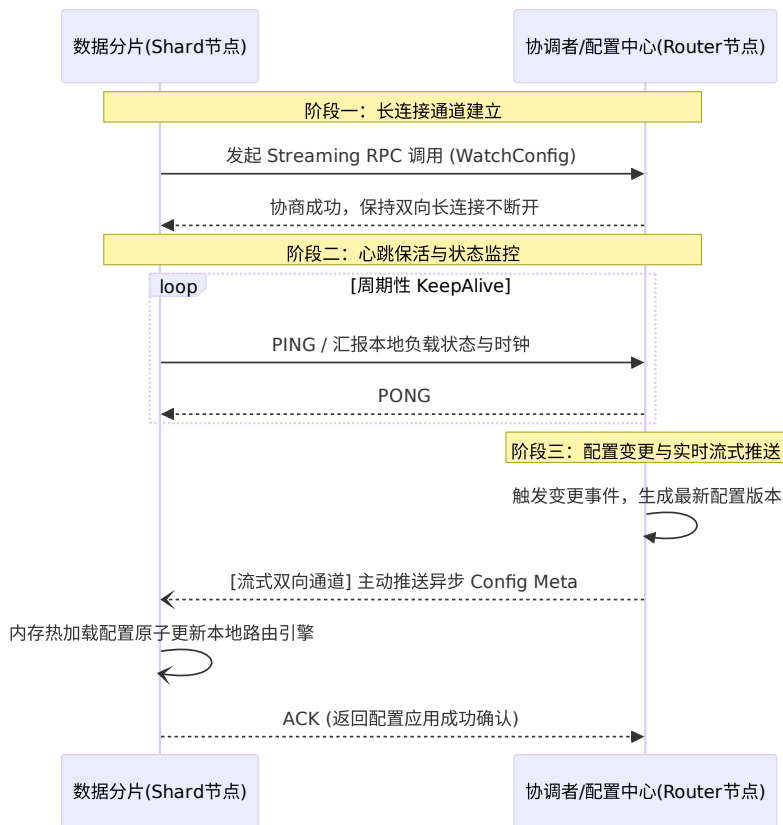


图 5-6 长连接配置同步示意图

长连接配置同步策略主要带来两方面作用。其一，当全局元数据发生变更时，Router 可以通过常驻的数据流通道主动下发增量或全量配置，从而减少轮询机制带来的同步延迟和无效探测开销。其二，该长连接通道同时承担心跳保活（KeepAlive）功能，节点间的 Ping-Pong 心跳有助于 Router 较快感知底层分片的宕机或网络异常，并据此执行路由重定向与故障摘除（Failover）。

#### 5.4.4 用户客户端和测试客户端

用户客户端的职责相对直接。客户端连接到路由节点（Router）后，用户可以通过命令行输入数据操作与事务管理命令。这里最关键的设计点不是交互形式，而是事务亲和性：同一事务的全部操作必须落到同一个 Router，才能保证事务上下文完整。基于这一要求，客户端只在事务开始时选择一次 Router；在该事

务结束之前，后续读写、提交和中止请求都沿用这条连接。事务结束后，客户端再为下一笔事务重新选择路由节点。

```
>start SER
Tx_mulberry-server-2_1709561234890> read
  ↳ account-1
<nil>
Tx_mulberry-server-2_1709561234890> write
  ↳ account-1 100
OK
Tx_mulberry-server-2_1709561234890> commit
Committed
```

图 5-7 用户客户端示例交互

图 5-7 展示了一个基于命令行交互的用户客户端示例。用户可以通过输入简单的命令来执行数据操作和事务管理，如开启事务、读写数据、提交或中止事务等。客户端会将用户输入的命令解析为相应的 RPC 请求，并发送到路由节点（Router）进行处理。开启事务后，客户端会提示事务的 ID，用户可以在事务上下文中继续执行读写操作，直到提交或中止事务。客户端还会显示操作的结果，如读取到的数据、写入是否成功、提交是否成功等。

在测试客户端（Test Client）方面，同一个事务的开启（Start）和后续操作（如 Read/Write）极有可能被路由到不同的路由节点（Router）。由于 Router 将事务上下文（活跃事务记录）维护在自身的进程内存中，这会导致后续节点因找不到事务上下文而抛出错误，破坏事务正确性。为解决该问题并满足高并发评测需求，测试客户端设计了基于 ClientPool 的连接与会话路由机制，其核心理念是将负载均衡的粒度从“每次 RPC”提升到了“每个事务”。该机制具备以下三个关键特性：

1. **事务亲和性（Transaction Affinity）**：客户端在发起 Start 请求前，从池中为其分配一个固定的 Client 实例。该事务内的全部后续操作以及最终的 Commit 或 Abort，均强制通过该 Client 收发，确保了一笔事务的所有 RPC 能够一致且完整地打到同一个 Router 上。事务终结后绑定关系解除。
2. **事务级负载均衡**：ClientPool 的分配调度利用原子计数器实现轮询逻辑，使高并发创建请求能够较均匀地分配到后端 Router。例如在 3 个 Router 和 10 个测试事务的场景下，各节点的承载比例大致为 4:3:3，从而减少明显的请

求倾斜。

3. **并发安全且无排他锁**：底层复用的 gRPC ClientConn 本身支持并发安全与多路复用。因此，不同事务可以并发使用池中的同一个 Client，而不必采用“借出-上锁-归还”的排他模型。这样可以避免客户端池大小直接成为测试并发度的主要限制。

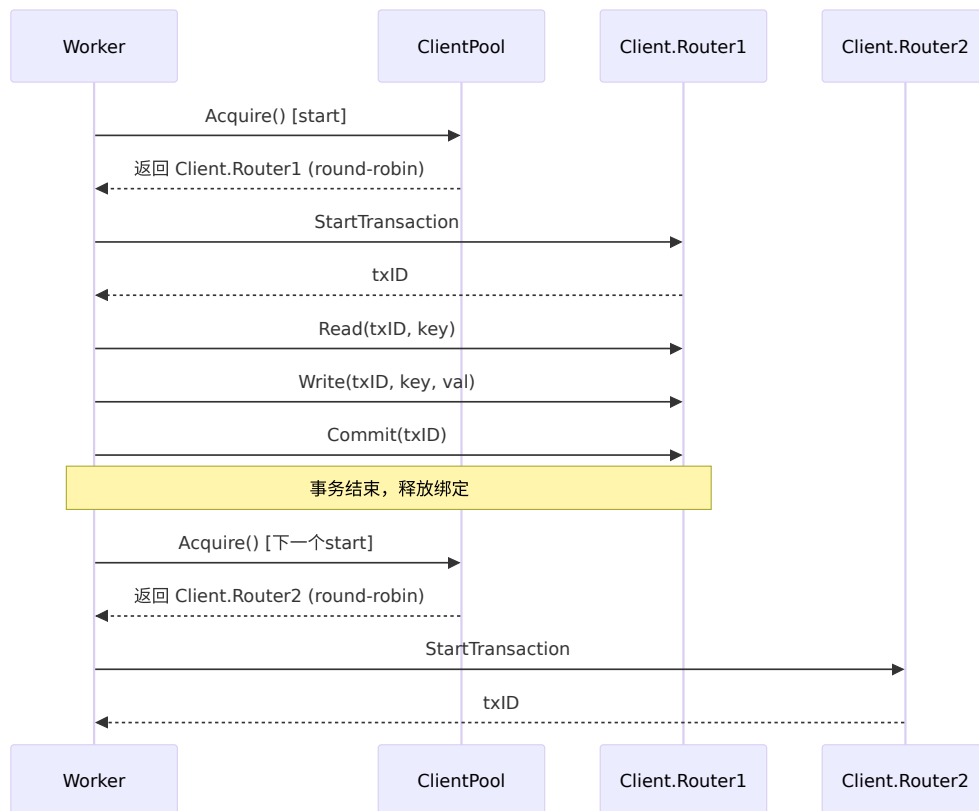


图 5-8 ClientPool 机制示意图

图 5-8 展示了基于 ClientPool 的机制。该设计在 gRPC 连接复用的基础上兼顾了事务路由的正确性与并发测试的实际需求。

测试用例通过配置文件的方式输入，测试客户端读取测试用例后，按照用例中定义的事务操作序列和隔离级别要求，自动执行。图 5-9 展示了一个测试配置文件的示例。第一行的数字指定 worker 的数量，worker 代表着每一个独立线程，随后定义每个 worker 要执行的事务的顺序和事务内部的操作细节。测试客户端会根据配置文件中的定义，自动创建相应数量的 worker。每个 worker 会按照配置文件中定义的事务操作序列，依次执行事务的开启、读写操作以及提交或中止等步骤。在执行过程中，测试客户端会记录每个操作的结果和性能指标，以便后

续分析和验证。

```

3
worker1:
start SI
write account_A 1000
write account_B 2000
write account_C 500
commit
start SI
read account_A
read account_B
write account_A 800
write account_B 2200
commit

worker2:
start SER
read account_A
read account_C
write account_C 300
commit
start SI
read account_A
read account_B
read account_C
delete account_C
commit

worker3:
start PSI
write order_1 pending
write order_2 pending
commit
start SER
read order_1
write order_1 confirmed
read order_2
write order_2 confirmed
commit

```

图 5-9 测试用例示例

#### 5.4.5 系统日志与操作可观测性

分布式数据库的性能调优与架构论证高度依赖于底层运行状态的透明化。本系统在分片（Shard）组件中内置了结构化的操作日志流（OpLog 与 NoOpLog 的灵活切换），并在事务执行的核心路径上无侵入地埋点了多维度的时间戳与状态指标。以此为基础，系统通过暴露核心指标构建了完整的可观测性监控体系，旨在以详实的实测数据支撑并验证系统在并发调度上的架构优势。

本项目的核心论证目标在于“探究混合隔离级别相比单一隔离级别在执行性能上的优势”。为了使这一命题可被量化，可观测性层面进行了针对性的精细粒度设计：系统中每一阶段的生命周期事件（包括跨分片的 Prepare 和 Commit，以及单分片优化的 QuickCommit）均会显式记录其绑定的隔离级别（IsolationLevel）元数据以及对应的操作耗时（RRT, Round-Trip Time）。不仅如此，系统还通过 LockManager 精确记录了高并发场景下悲观锁（尤其是 SER 隔离级别引发的共享读锁与排他写锁）排队阶段的跨事务真实锁等待耗时（Lock

Wait Time)；同时在乐观并发控制（如 SI 级别）通过版本比对的 CAS 执行域内，也对因写写冲突导致回滚（Abort Rate）和触发由于遇到 Pending 标志而被异常拦截的读取（ErrPendingRead 拦截率）进行了频率统计。

通过对上述多源探针数据的采集、聚合与可视化，系统能够较细致地呈现不同并发负载下的执行过程。这些监控数据表明，在混合隔离负载下，将核心事务保持为 SER，而将后台批处理或只读分析事务放宽至 SI 或 PC 后，系统可以利用 MVCC 的历史版本读取机制减少部分由强隔离带来的锁争用与排队开销。结合基准测试结果可以看出，混合隔离配置在当前原型系统中与更高吞吐量和较低排队开销相对应。

## 5.5 本章小结

本章说明了原型系统的定位、需求边界以及关键实现。系统采用分布式架构，支持 PC、PSI、SI 和 SER 四种隔离级别混合运行，并围绕全内存 MVCC 存储、基于 HLC 的时间定序、跨分片事务协调和客户端路由机制给出了具体实现。该原型为下一章的系统测试提供了可运行的实验平台。



## 第六章 基于分布式键值数据库原型的系统测试

本章在前述原型系统上测试不同隔离级别配置下的事务执行性能，重点观察混合隔离级别机制相较于统一强隔离方案的吞吐量变化。下文依次介绍测试环境、部署方式与评价指标，并给出三组数据集在两种部署拓扑下的实验结果。

### 6.1 测试环境概述

数据库原型系统基于 Golang 开发，并通过 Docker 容器模拟分布式数据库集群。测试均在本地环境中完成，容器的 CPU 与内存配额保持一致，以避免资源分配差异对结果造成额外干扰。除测试客户端外，集群共部署 6 个服务节点，包括 2 个 Router 节点、3 个 Shard 节点和 1 个 Config 节点，其中 Config 节点主要负责元数据管理。具体硬件与软件配置见表 6-1。

本文重点考察两种部署方式。第一种方式中，6 个服务节点全部部署在同一可用区内；第二种方式中，6 个服务节点按每两个节点一组分散部署到 3 个可用区中。两种部署方式使用相同的事务负载与资源限制，因此测试结果的差异主要反映节点分散程度对分布式事务协调开销的影响。

表 6-1 本地测试环境配置

| 组件            | 配置参数                                    |
|---------------|-----------------------------------------|
| 处理器 (CPU)     | AMD Ryzen 5 5600X (6 Cores, 12 Threads) |
| 内存 (RAM)      | 32GB DDR4 3600MHz                       |
| 操作系统 (OS)     | Windows 11 Home 25H2                    |
| 容器资源限制        | 1 vCPU, 2GB RAM                         |
| 编译环境 (Golang) | 1.24-alpine                             |
| 容器基础镜像        | alpine:3.19                             |

图 6-1展示了选取在 SER 隔离级别下执行 TPCC 负载，系统资源的水位图和事务延迟情况，可以看到在测试过程中系统资源使用率保持在合理范围内，且事务响应延迟稳定，即使触发 GC，延迟也在预期水平。这表明测试环境能够承载

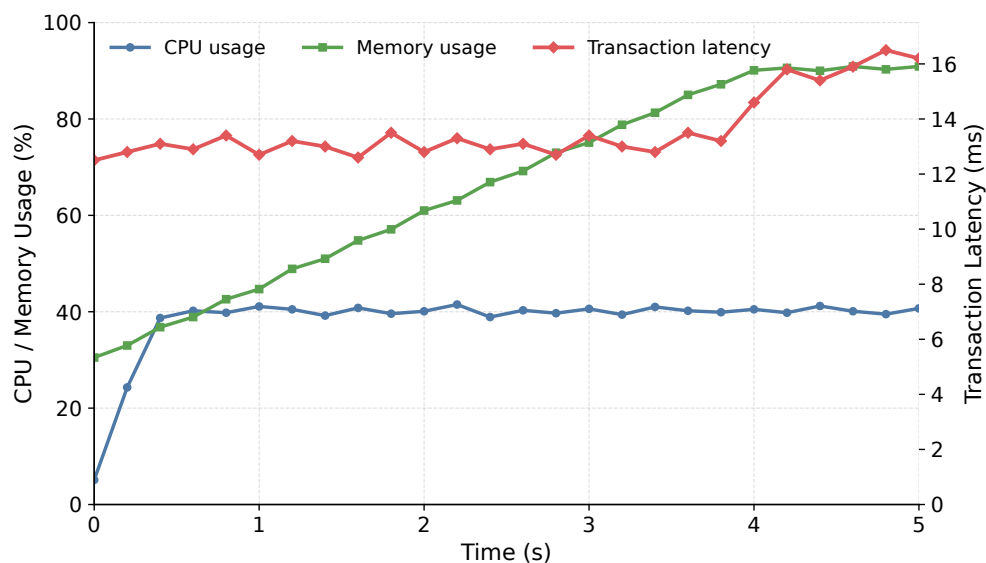


图 6-1 系统资源水位与响应延迟变化图

后续的隔离级别性能对比实验。

## 6.2 测试负载与评价指标

为了覆盖不同类型的事务访问模式，实验选取了 Courseware、SmallBank 和 TPCC 三个数据集，分别对应课程管理、银行转账和订单处理等场景。三组负载在读写比例、热点分布和事务冲突特征上存在差异，因此适合用来观察混合隔离级别机制在不同业务模式下的表现。

测试中比较了三种隔离级别配置。SER 表示所有事务均采用可串行化隔离级别，SI-SER 表示事务在快照隔离与可串行化之间混合执行，MIAED 则表示引入 MIA 策略后的混合隔离级别分配结果。MIA 是本组的另一项工作，主要根据事务内容分析结果，在保证事务正确性的前提下尽量匹配满足要求的最低隔离级别。本文将 MIAED 作为扩展性对照，用于观察更细粒度事务分配策略下系统性能的变化；本章的重点仍是考察本文实现的混合隔离级别执行机制，相较于统一强隔离基线能否带来性能改善。评价指标以平均吞吐量（transactions per second, tx/s）为主，用于衡量系统在稳定运行阶段的整体处理能力；同时，本文补充统计了系统运行过程中因写写冲突或提交阶段校验失败而触发的事务回滚率（Abort Rate），以辅助分析不同负载与部署方式下的并发冲突程度。由于当前回滚率数据覆盖的是 SER 与 MIAED 两组配置，后文相关分析主要围绕这两类

策略展开。

## 6.3 测试结果与分析

图 6-2 展示了三组数据集在两种部署方式下、不同隔离级别配置中的吞吐量结果。

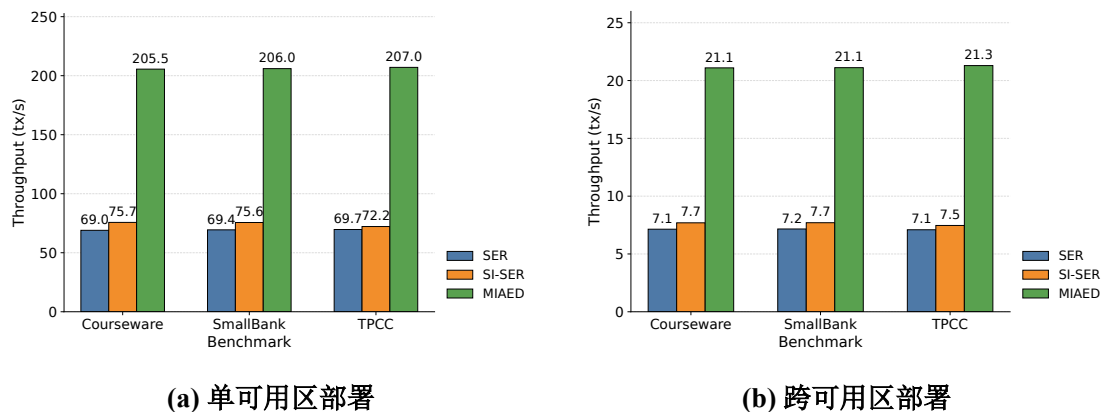


图 6-2 不同部署方式下各隔离级别配置的吞吐量对比

在单可用区部署下，SER 的平均吞吐量为 69.36 tx/s，SI-SER 为 74.51 tx/s，平均增幅约为 7.42%。也就是说，即使只在 SI 和 SER 两个较强隔离级别之间做有限混合，系统也能释放一部分并发能力。作为扩展性对照，MIAED 的平均吞吐量达到 206.19 tx/s，较 SER 提升约 197.28%，较 SI-SER 提升约 176.74%。这一趋势在三组数据集上都较稳定：Courseware、SmallBank 和 TPCC 上相对 SER 的提升分别为 197.94%、196.81% 和 197.10%。结合部署条件看，单可用区下事务协调路径较短，网络往返等待较小，因此更容易体现混合执行带来的吞吐收益。

当节点分散到多个可用区后，整体吞吐量明显下降，但不同策略之间的相对关系没有发生明显变化。SER 的平均吞吐量降至 7.13 tx/s，SI-SER 为 7.62 tx/s，平均增幅约为 6.85%。这说明即使跨区部署显著抬高了协调开销，SI-SER 相对 SER 仍保留了稳定但有限的吞吐优势。作为扩展性对照，MIAED 的平均吞吐量为 21.17 tx/s，较 SER 提升约 196.81%，较 SI-SER 提升约 177.78%。分数据集来看，Courseware、SmallBank 和 TPCC 上相对 SER 的提升分别为 195.40%、194.87% 和 200.18%。因此，跨区通信虽然放大了协调成本，但更细粒度的事务分配策略依旧能够取得更高吞吐量。

横向比较两种部署方式可以看到，当节点由单可用区集中部署转为跨可用

区分散部署后，三种策略的平均吞吐量都降到原来的约 10%：SER、SI-SER 和 MIAED 的降幅分别为 89.72%、89.77% 和 89.73%。这一结果说明，节点分散后事务提交路径被拉长，跨节点协调与通信等待成为主导开销。不过，SI-SER 相对 SER 的优势在两类部署方式中都能观察到，说明这种混合执行方式并不依赖某一种特定拓扑；MIAED 的额外收益则提示，更细粒度的事务分配仍有进一步优化空间。

为了更直观地观察混合策略相对统一强隔离基线的收益，图 6-3 进一步给出了各策略相对 SER 的吞吐提升百分比。

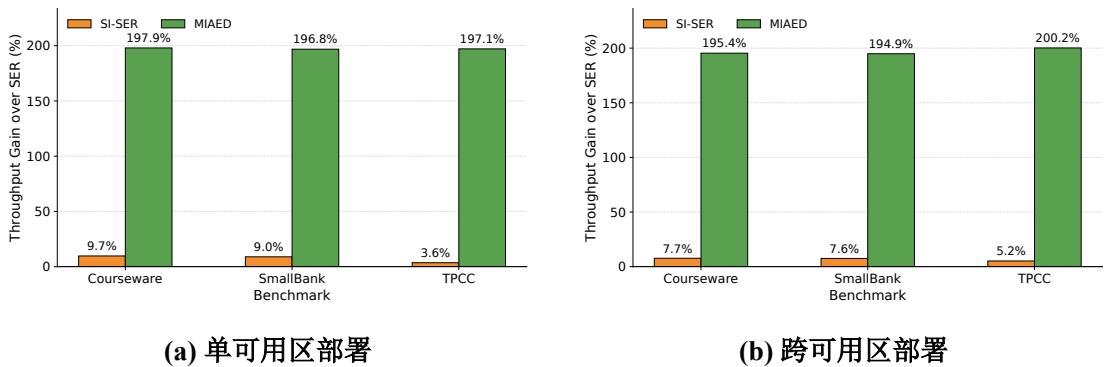


图 6-3 不同部署方式下各策略相对 SER 的吞吐提升百分比

从图 6-3 可以更直接地看出，SI-SER 相对 SER 的收益虽然不大，但在两种部署方式和三组数据集上始终为正，提升区间约为 3.61%–9.73%。这说明仅在 SI 和 SER 两级之间进行混合，也能回收一部分因统一串行化而损失的并发能力。相比之下，MIAED 相对 SER 的提升接近 2 倍，单可用区下为 196.81%–197.94%，跨可用区下为 194.87%–200.18%。因此，这组结果实际上对应两类不同幅度的收益：前者来自有限的隔离级别混合，后者则来自更细粒度的事务分配。

为了进一步观察不同负载下事务冲突对执行结果的影响，本文补充统计了 SER 与 MIAED 两组配置在两种部署方式下的事务回滚率，结果如图 6-4 所示。

从图 6-4 可以看出，不同负载的冲突强度存在明显差异。Courseware 的回滚率最低，单可用区下 SER 和 MIAED 分别为 1.1% 和 1.7%，跨可用区下升至 2.1% 和 2.5%，整体仍处于较低水平，说明该负载中的热点写入较少。TPCC 的回滚率居中，单可用区下为 5.0% 和 6.8%，跨可用区下为 6.7% 和 9.2%。SmallBank 的回滚率最高，单可用区下 SER 与 MIAED 已分别达到 10.1% 和 12.7%，跨可用区下进一步上升到 13.5% 和 16.2%。这一结果与三类负载的事务特征基本一致：

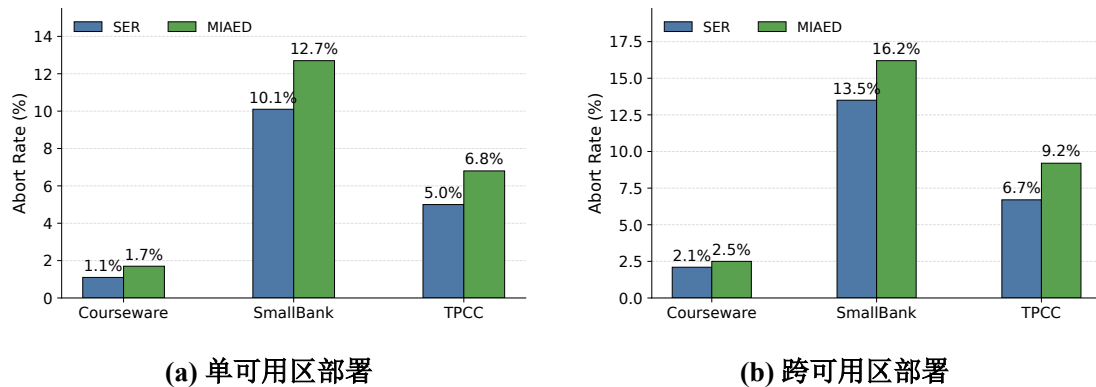


图 6-4 不同部署方式下 SER 与 MIAED 的事务回滚率对比

Courseware 的更新分布较分散，TPCC 存在一定程度的热点竞争，而 SmallBank 中账户读写更集中，因而更容易在提交校验阶段触发事务中止。

若比较同一负载在两种部署方式下的结果，可以发现跨可用区部署会系统性抬高回滚率。以 SER 为例，Courseware、SmallBank 和 TPCC 分别从 1.1%、10.1% 和 5.0% 上升到 2.1%、13.5% 和 6.7%；MIAED 也分别从 1.7%、12.7% 和 6.8% 上升到 2.5%、16.2% 和 9.2%。这说明跨区通信引入的额外时延会拉长事务从读取到提交的时间窗口，从而增加并发事务之间发生冲突的概率。另一方面，MIAED 在所有负载上都表现出高于 SER 的回滚率，且在高冲突负载上的差距更明显，例如 SmallBank 下两种部署方式分别高出 2.6 和 2.7 个百分点，TPCC 下分别高出 1.8 和 2.5 个百分点。这表明更积极的并发执行策略确实会带来一定的冲突代价；但结合前文吞吐量结果看，这一代价换来了更高的系统处理能力，因而整体上仍体现出较好的性能收益。

综合三组数据，可以得到两个观察。第一，单一 SER 虽然提供了更强的一致性保证，但其保守的并发控制会降低系统吞吐量；将部分事务放宽到 SI-SER 后，可以获得有限但较为稳定的性能提升。第二，MIAED 作为扩展性对照，在三个数据集和两种拓扑下均取得了最高吞吐量，这说明如果在本文机制之上进一步引入更细粒度的事务分类与分配策略，系统性能仍有继续提升的空间。

## 6.4 本章小结

本章比较了 Courseware、SmallBank 和 TPCC 三组负载在两种部署方式下的吞吐量与回滚率，并对 SER、SI-SER 和 MIAED 三种配置进行了分析。实验结

果表明，跨可用区部署会显著拉低整体吞吐量，但 SI-SER 相对 SER 的优势在两类拓扑下都能够观察到，说明本文实现的混合隔离级别机制确实减少了部分不必要的强同步开销。回滚率结果则说明，不同负载的冲突特征差异明显，且跨区部署会放大事务中止概率；与此同时，MIAED 在获得更高吞吐量的同时，没有带来同幅度的额外冲突代价。

## 第七章 工作总结与展望

### 7.1 工作总结

本文讨论的问题是：分布式数据库如何在保证事务语义正确的前提下支持混合隔离级别，并进一步提升整体执行效率。现有工作往往分别停留在语义分析、形式化验证或系统实现中的某一环节，三者之间衔接不够紧密。本文的重点，是把语义定义、协议设计、形式化验证、原型实现和实验评估串联起来。

在理论层面，本文围绕事务异常、快照语义与一致性约束，对分布式事务执行过程进行了统一描述。通过引入抽象执行模型，本文对事务历史、可见性关系 VIS、仲裁关系 AR 以及相应一致性公理进行了形式化定义，并在此基础上讨论了 RA、CC、PC、PSI、SI 和 SER 等隔离级别的统一表达方式。进一步地，本文将研究重点扩展到混合隔离级别场景，指出当不同事务在同一执行历史中采用不同隔离级别时，系统正确性不能被理解为各事务局部正确性的简单叠加，而需要在统一抽象执行下进行全局判定。这一结论为后续系统设计提供了明确的语义基础。

在系统设计与验证层面，本文面向支持混合隔离级别的分布式键值数据库，给出了一套与抽象执行语义相对应的操作语义，并使用  $TLA^+$  与 TLC 对关键安全性质进行了建模和验证。该设计以客户端、路由节点、配置节点和分片节点组成的分布式架构为基础，引入 HLC 作为全局定序工具，以 MVCC 支撑快照读取与版本管理，并结合依赖追踪、等待机制、检查集维护、冲突检测以及两阶段提交等协议机制，将不同隔离级别所要求的可见性与顺序约束落实到可执行的状态转移规则中。通过形式化验证，本文说明了在给定有限模型范围内，系统所允许的状态演化不会产生违反目标一致性约束的执行，从而在抽象语义与具体协议之间建立了可验证的联系。

在工程实现与实验评估层面，本文设计并实现了一个支持混合隔离级别的分布式键值数据库原型系统。该原型采用 Router-Shard 架构，集成了分片管理、

路由转发、HLC 时间戳维护、MVCC 存储、多分片事务协调以及单分片快速提交等核心机制，支持 PC、PSI、SI 和 SER 四种隔离级别在同一系统中的混合运行。基于 Courseware、SmallBank 和 TPCC 三组负载，本文在单可用区集中部署和跨可用区分散部署两种拓扑下，对不同隔离级别配置进行了对比测试。实验结果表明，相较于统一采用强隔离级别的基线配置，本文所实现的混合隔离级别执行机制能够在保证目标语义成立的前提下稳定提升系统吞吐量；即使在跨可用区部署、通信开销明显上升的条件下，这种相对优势仍然能够保持。扩展性对照实验还表明，在本文机制之上进一步引入更细粒度的事务分配策略，系统性能仍存在继续提升的空间。

综合来看，本文的工作可以归纳为三点：一是为混合隔离级别场景建立了统一的一致性语义与抽象执行判定框架；二是设计并验证了一套面向分布式键值数据库的混合隔离级别操作语义；三是通过原型实现和实验评测，说明了该机制在给定实验条件下具备工程可实现性和一定的性能收益。本文并未穷尽这一方向的全部问题，但至少给出了一条从语义到实现再到实验验证的较完整路径。

## 7.2 未来展望

本文的工作仍然带有明显的研究型原型特征。受限于时间和实现范围，一些问题只做到了初步展开，后续仍有继续深入的空间。

首先，在形式化建模与验证范围方面，本文的  $TLA^+$  模型主要聚焦于事务可见性、提交顺序和隔离级别约束等核心语义，对故障恢复、副本复制、网络分区、节点动态扩缩容等工程因素进行了必要抽象。这种抽象有助于突出本文关注的主要问题，但也意味着当前验证结果主要针对理想化或受控条件下的系统行为。未来可在保持模型可分析性的前提下，进一步引入故障场景、恢复过程与副本一致性约束，研究混合隔离级别机制在更复杂分布式环境中的正确性边界。

其次，在系统实现层面，本文原型系统仍以研究验证为主要目标，尚未覆盖生产级数据库所需的全部能力。例如，当前系统对持久化、日志恢复、复制容错、在线重配置与弹性伸缩等关键机制的支持仍不完整；部分模块采用了有利于研究验证的简化设计，而非面向大规模部署的实现方案。未来工作可围绕存储引擎、日志子系统和复制协议进行进一步完善，并评估不同底层设计对混合隔离级

别机制的影响，以增强系统的稳定性、可扩展性和工程适用性。

再次，在事务协调机制方面，本文对跨分片事务原子性的保证主要依赖两阶段提交协议（2PC）。这一方案实现清晰、便于分析，但其阻塞特性与额外协调开销会在高并发、广域网络或参与分片较多的场景下放大系统延迟，并对可用性造成一定影响。未来可进一步研究更高效的原子提交机制，或者探索与复制协议、时间戳定序机制更紧密结合的事务协调方法，以降低跨分片提交的等待成本，并提升系统在复杂部署条件下的整体表现。

此外，本文实验评测虽然已经覆盖多种工作负载与两类部署拓扑，但整体上仍以原型验证为主，评测范围和指标维度仍可扩展。一方面，未来可引入更加标准化和更大规模的基准测试，对系统在不同读写比例、事务长度、热点分布与跨分片比例下的行为进行更细致分析；另一方面，也可进一步关注事务中止率、长尾延迟、资源利用率以及隔离级别配置对不同业务类型的影响，从而更加全面地评估混合隔离级别机制在正确性与性能之间的权衡效果。

最后，从更长期的研究方向看，混合隔离级别机制本身仍有较大的拓展空间。本文主要关注固定隔离级别集合下的统一语义与系统支持，而在真实业务中，事务的隔离需求往往和操作模式、冲突特征、业务约束一起决定。后续可以进一步研究隔离级别推荐与自动配置问题，并将其与本文建立的统一语义框架结合起来，使系统不仅支持多种隔离级别共存，还能更有针对性地为不同事务选择执行策略。

总体来看，混合隔离级别分布式数据库仍然是一个兼具理论挑战和工程价值的研究方向。本文的工作还不足以覆盖这一方向的全部关键问题，但相关分析、建模与实验结果说明，围绕统一语义框架、可验证协议设计和工程实现协同推进，是一条值得继续深入的路径。



## 参考文献

- [1] 陈志标, 欧伟杰, 孟凡彬, 等. 迎接人工智能挑战: 构建下一代数据库系统[J]. 计算, 2025, 1(7): 78-88.
- [2] CATTELL R. Scalable SQL and NoSQL data stores[J]. SIGMOD Record, 2011, 39(4): 12-27.
- [3] STONEBRAKER M. SQL databases v. NoSQL databases[J]. Communications of the ACM, 2010, 53(4): 10-11.
- [4] AGRAWAL D, EL ABBADI A, EMAM A. The evolution of cloud computing [J]. IEEE Cloud Computing, 2016, 3(2): 70-73.
- [5] ÖZSU M T, VALDURIEZ P. Principles of distributed database systems[M]. 3rd. Springer Science & Business Media, 2011.
- [6] DRAGONI N, GIALLORENZO S, LAFUENTE A L, et al. Microservices: yesterday, today, and tomorrow[J]. Present and ulterior software engineering, 2017: 195-216.
- [7] CORBETT J C, DEAN J, EPSTEIN M, et al. Spanner: Google' s globally distributed database[J]. ACM Transactions on Computer Systems (TOCS), 2013, 31(3): 1-22.
- [8] GRAY J, REUTER A. Transaction processing: concepts and techniques[M]. Morgan Kaufmann, 1993.
- [9] BERENSON H, BERNSTEIN P, GRAY J, et al. A Critique of ANSI SQL Isolation Levels[C/OL]//Proceedings of the 1995 ACM SIGMOD International Conference on Management of Data. 1995: 1-10. DOI: 10.1145/223784.223785.
- [10] HAERDER T, REUTER A. Principles of transaction-oriented database recovery [J]. ACM Computing Surveys (CSUR), 1983, 15(4): 287-317.

- [11] BREWER E A. Towards robust distributed systems[C]//Proceedings of the 19th Annual ACM Symposium on Principles of Distributed Computing (PODC). 2000: 7.
- [12] GILBERT S, LYNCH N. Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services[J]. SIGACT News, 2002, 33(2): 51-59.
- [13] BAILIS P, DAVIDSON A, FEKETE A, et al. Highly available transactions: Virtues and limitations[J]. Proceedings of the VLDB Endowment, 2013, 7(3): 181-192.
- [14] CROOKS N, PU Y, ALVISI L, et al. Seeing is believing: A client-centric specification of database isolation[C]//Proceedings of the ACM Symposium on Principles of Distributed Computing. 2017: 73-82.
- [15] TERRY D B, PRABHAKARAN V, KOTLA R, et al. Consistency-based service level agreements for cloud storage[C]//Proceedings of the 24th ACM Symposium on Operating Systems Principles (SOSP). 2013: 309-324.
- [16] BAILIS P, VENKATARAMAN S, FRANKLIN M J, et al. Probabilistic Bounded Staleness (PBS) for Read-Only Transactions[J]. Proceedings of the VLDB Endowment, 2012, 5(11): 1774-1775.
- [17] TAFT R, SHARIF I, MATEI A, et al. Cockroachdb: The resilient geo-distributed sql database[C]//Proceedings of the 2020 ACM SIGMOD international conference on management of data. 2020: 1493-1509.
- [18] CERONE A, BERNARDI G, GOTSMAN A. A framework for transactional consistency models with atomic visibility[C]//26th International Conference on Concurrency Theory (CONCUR 2015). 2015: 58-71.
- [19] BOUAJJANI A, ENEA C, ROMÁN-CALVO E. On the Complexity of Checking Mixed Isolation Levels for SQL Transactions[C]//International Conference on Computer Aided Verification. 2025: 315-337.
- [20] SCHULTZ W, DEMIRBAS M. Design and Modular Verification of Distributed Transactions in MongoDB[J/OL]. Proc. VLDB Endow., 2025, 18(12): 5045-

5058. <https://doi.org/10.14778/3750601.3750626>. DOI: 10.14778/3750601.3750626.
- [21] HAWBLITZEL C, HOWELL J, KAPRITSOS M, et al. IronFleet: proving practical distributed systems correct[C]//Proceedings of the 25th Symposium on Operating Systems Principles. 2015: 1-17.
- [22] LAMPSON B, STURGIS H. Crash recovery in a distributed data storage system[C]//Distributed Systems: Architecture and Implementation. Springer, 1976: 357-370.
- [23] PERKINS E, WONG P, GUPTA I. Performance analysis of distributed transactions in partitioned database systems[C]//Proceedings of the 2011 IEEE International Symposium on Modeling, Analysis and Simulation of Computer and Telecommunication Systems. 2011: 250-259.
- [24] CARLSON J L. Redis in Action[M]. Manning Publications Co., 2013.
- [25] PAVLO A, ASLETT M. What's really new with NewSQL?[J]. SIGMOD Record, 2016, 45(2): 45-55.
- [26] DECANDIA G, HASTORUN D, JAMPANI M, et al. Dynamo: amazon's highly available key-value store[C]//Proceedings of twenty-first ACM SIGOPS symposium on Operating systems principles (SOSP). 2007: 205-220.
- [27] LAKSHMAN A, MALIK P. Cassandra: a decentralized structured storage system[J]. SIGOPS Operating Systems Review, 2010, 44(2): 35-40.
- [28] VOGELS W. Eventually consistent[J]. Communications of the ACM, 2009, 52(1): 40-44.
- [29] HARDING R, VAN AKEN D, PAVLO A, et al. An evaluation of distributed concurrency control[J]. Proceedings of the VLDB Endowment, 2017, 10(5): 553-564.
- [30] THOMSON A, ABADI D J, LAHIRI A, et al. Calvin: Fast distributed transactions for partitioned database systems[C]//Proceedings of the 2012 ACM SIGMOD International Conference on Management of Data. 2012: 1-12.

- [31] NEWCOMBE C, RATH T, ZHANG F, et al. How Amazon web services uses formal methods[J]. Communications of the ACM, 2015, 58(4): 66-73.
- [32] BAILIS P, FEKETE A, GHODSI A, et al. Scalable Atomic Visibility with RAMP Transactions[J/OL]. ACM Transactions on Database Systems (TODS), 2016, 41(3): 15:1-15:45. DOI: 10.1145/2909870.
- [33] AKKOORATH D D, TOMSIC A Z, BRAVO M, et al. Cure: Strong Semantics Meets High Availability and Low Latency[C/OL]//2016 IEEE 36th International Conference on Distributed Computing Systems (ICDCS). 2016: 405-414. DOI: 10.1109/ICDCS.2016.98.
- [34] SOVRAN Y, POWER R, AGUILERA M K, et al. Transactional Storage for Geo-replicated Systems[C/OL]//Proceedings of the Twenty-Third ACM Symposium on Operating Systems Principles (SOSP). 2011: 385-400. DOI: 10.1145/2043556.2043592.
- [35] BURCKHARDT S, LEIJEN D, PROTZENKO J, et al. Global sequence protocol: A robust abstraction for replicated shared state[C]//29th European Conference on Object-Oriented Programming (ECOOP 2015). 2015: 568-590.
- [36] PAPADIMITRIOU C H. The Serializability of Concurrent Database Updates [J/OL]. Journal of the ACM (JACM), 1979, 26(4): 631-653. DOI: 10.1145/322154.322158.
- [37] TERRY D B, PRABHAKARAN V, KOTLA R, et al. Consistency-based service level agreements for cloud storage[C]//Proceedings of the twenty-fourth ACM symposium on operating systems principles. 2013: 309-324.
- [38] STONEBRAKER M, et al. The case for shared nothing[J]. IEEE Database Eng. Bull., 1986, 9(1): 4-9.
- [39] FALEIRO J M, ABADI D J. Rethinking serializable multiversion concurrency control[J/OL]. Proc. VLDB Endow., 2015, 8(11): 1190-1201. <https://doi.org/10.14778/2809974.2809981>. DOI: 10.14778/2809974.2809981.
- [40] SKEEN D, STONEBRAKER M. A formal model of crash recovery in a dis-

- tributed system[J]. IEEE Transactions on Software Engineering, 1983(3): 219-228.
- [41] KULKARNI S S, DEMIRBAS M, MADAPPA D, et al. Logical Physical Clocks and Consistent Snapshots in Globally Distributed Databases: 2014-04[R/OL]. University at Buffalo, Department of Computer Science. 2014. <https://cse.buffalo.edu/tech-reports/2014-04.pdf>.
- [42] PENG D, DABEK F. Large-scale incremental processing using distributed transactions and notifications[C]//9th USENIX Symposium on Operating Systems Design and Implementation (OSDI 10). 2010.
- [43] ESWARAN K P, GRAY J N, LORIE R A, et al. The notions of consistency and predicate locks in a database system[J]. Communications of the ACM, 1976, 19(11): 624-633.
- [44] KULKARNI S S, DEMIRBAS M, MADAPPA D, et al. Logical physical clocks [C]//International Conference on Principles of Distributed Systems. 2014: 17-32.
- [45] DEWITT D, GRAY J. Parallel database systems: The future of high performance database systems[J]. Communications of the ACM, 1992, 35(6): 85-98.
- [46] THOMSON A, DIAMOND T, WENG S C, et al. Calvin: fast distributed transactions for partitioned database systems[C/OL]//SIGMOD '12: Proceedings of the 2012 ACM SIGMOD International Conference on Management of Data. Scottsdale, Arizona, USA: Association for Computing Machinery, 2012: 1-12. <https://doi.org/10.1145/2213836.2213838>. DOI: 10.1145/2213836.2213838.
- [47] BAILIS P, FEKETE A, FRANKLIN M J, et al. Coordination avoidance in database systems[J/OL]. Proc. VLDB Endow., 2014, 8(3): 185-196. <https://doi.org/10.14778/2735508.2735509>. DOI: 10.14778/2735508.2735509.



## 附录 A TLA<sup>+</sup> 模型检查补充材料

### A.1 SmallBank 负载测试用例

```

MODULE Benchmark_SmallBank
EXTENDS Naturals, Sequences

Keys    ≜ { "checking", "savings" }
Values  ≜ { 0, 1, 2 }
InitVal ≜ 0

We define the Workload as a Sequence (Tuple) of Records.
Workload ≜ ⟨ T1: TransactSavings (PSI)
[
  session ↦ "client_1",
  level   ↦ "PSI",
  ops     ↦ ⟨ [type ↦ "R", key ↦ "checking"],
              [type ↦ "R", key ↦ "savings"], [type ↦ "W", key ↦ "savings", val ↦ 1] ⟩
], T2: WriteCheck (SER)
[
  session ↦ "client_2",
  level   ↦ "SER",
  ops     ↦ ⟨ [type ↦ "R", key ↦ "checking"],
              [type ↦ "R", key ↦ "savings"], [type ↦ "W", key ↦ "checking", val ↦ 2] ⟩
]
⟩

VARIABLES
  client_dep,
  client_last_commit,
  router_ct,
  shard_ct,
  tx_status,
  tx_snapshot,
  tx_checkset,
  tx_buffer,
  tx_commit_time,
  pc,
  store,
  op_seq

MI ≜ INSTANCE MixedIsolation WITH
  Keys ← Keys,
  Values ← Values,
  InitVal ← InitVal,
  Workload ← Workload

Spec ≜ MI!Spec
OperationalSafety ≜ MI!OperationalSafety

```

图 A-1 SmallBank 负载测试用例